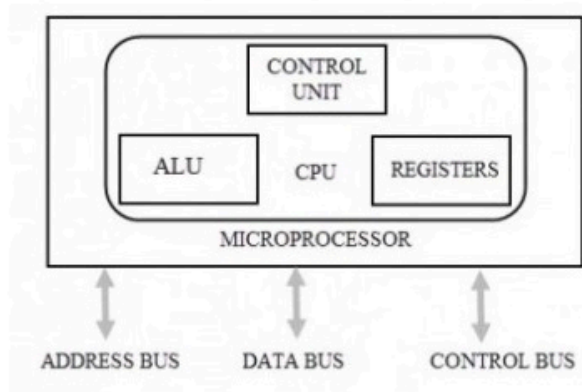


Microprocessors Q & A's:

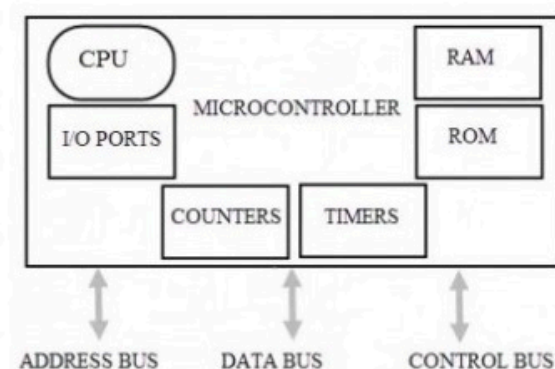
1. Difference between microprocessor and microcontroller and what is Von-Newman architecture?
8086 and 8088 microprocessors.

<u>MICROPROCESSOR</u>	<u>MICROCONTROLLER</u>
Center of a computer system.	Center of embedded system.
Memory and I/O components are external to it.	Memory and I/O components are internal to it.
Large Circuit	Smaller Circuit
Not compatible with compact systems	Compatible with compact systems.
Higher cost	Lower Cost
High Power Consumption	Low Power Consumption
Mostly don't have power features	Mostly have power features.
Mainly present in personal computers.	Mainly present in washing machines, music players, and embedded systems.
Less number of registers.	More number of registers.
Follows Von Neumann model	Follows Harvard architecture
Made on a silicon-based integrated chip.	Byproduct microprocessors and peripherals.
RAM, ROM, and other peripherals are absent.	RAM, ROM, and other peripherals are present.
Has an external bus to interface with devices.	Uses an internal controlling bus for communication.
Has a high speed.	Speed depends on the architecture.
Ideal for general purpose to handle more data.	Ideal for the specific applications.
Complex and Expensive	Simple and affordable
Requires more instructions	Requires less instructions

Microprocessor



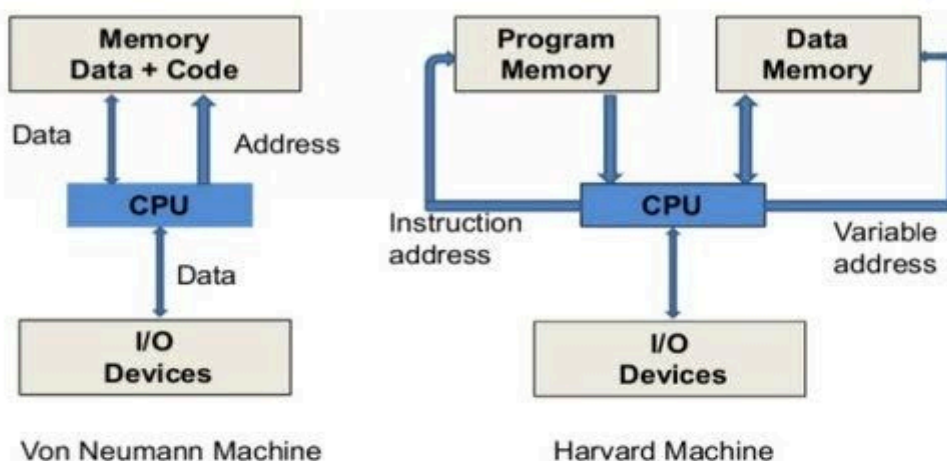
Microcontroller



VON NEUMANN ARCHITECTURE VERSUS HARVARD ARCHITECTURE

It is a theoretical design based on the stored-program computer concept.	It is a modern computer architecture based on the Harvard Mark I relay-based computer model.
It uses same physical memory address for instructions and data.	It uses separate memory addresses for instructions and data.
Processor needs two clock cycles to execute an instruction.	Processor needs one cycle to complete an instruction.
Simpler control unit design and development of one is cheaper and faster.	Control unit for two buses is more complicated which adds to the development cost.
Data transfers and instruction fetches cannot be performed simultaneously.	Data transfers and instruction fetches can be performed at the same time.
Used in personal computers, laptops, and workstations.	Used in microcontrollers and signal processing.

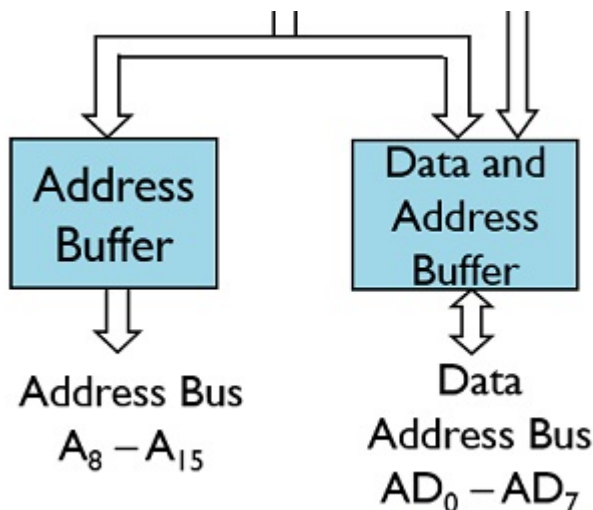
Von Neumann vs. Harvard Architecture



COMPARISON OF 8086 & 8088	
8086	8088
• It has 16 bit data bus • It can read or write 8/16 bit data at a time • Memory space is organized as two 512KB (2×512KB = 1 MB) banks • It can operate at three clock speed , ie ; 5MHz , 8MHz and 10MHz • Instruction queue is 6 bit long • It has BHE pin • It draws maximum supply current of 360MA	• It has 8 bit data bus • It can only do so for 8 bit data • It is implemented as a single 1MB bank • It is available only in two clock speed , ie; 5MHz and 8MHz • Instruction queue is 4 bit long • This pin is replaced by status output (SSO), since it can read or write only 8 bit data • It draws maximum supply current of 340MA

2. Explain the demultiplexing of the address/data multiplexed bus with the help of proper diagrams. Explain the function of ALE pin of 8085 microprocessor.

The signal lines AD0 to AD7 in an 8085 microprocessor used for dual purpose. They carry address bits as well as data bits. This is called multiplexed address/data bus. This bus is used as a low order address bus and data bus. The address bus and data bus are multiplexed to reduce the number of pins in the IC package.

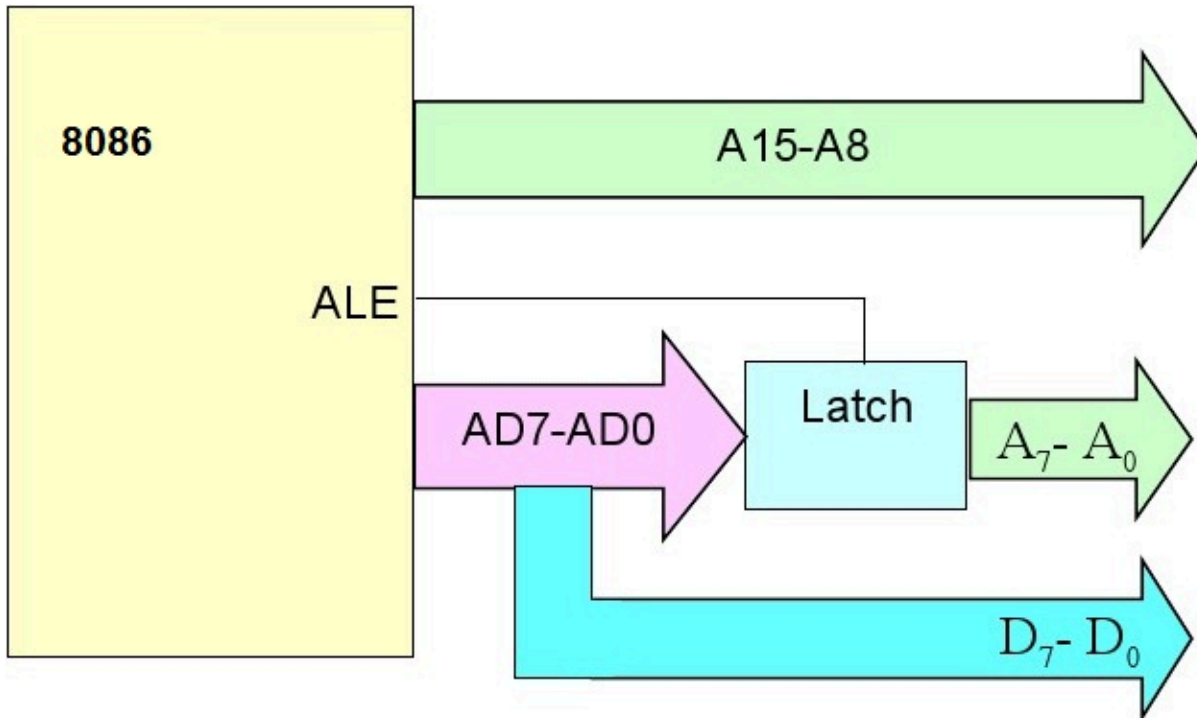


Latch To avoid mixing up of address and data bits this must be de-multiplexed. This can be done with the help of a latch (74LS373). This bus is connected as input to the latch.

Address Latch Enable (ALE) The Address Latch Enable (ALE) signal is connected to the enable pin (G) of the latch and output control (OC) pin is grounded.

When ALE goes high, the latch is transparent and it enables lower order address bits (A0 to A7).

When ALE goes low, the latch enables lower order data bits (D0 to D7)



In the 8085 system, memory of a group of 8 bit (1 Byte) has a specific address. The address is of the length 16 bit. So it requires a 16 bit address bus, and an 8 bit data bus to fetch data from the memory. In the place of making the data bus separate, 8085 uses a low order address bus shared with the data bus by method of multiplexing. This is done to reduce the number of pins. So, AD0-AD7 is an address and data bus at different times.

ALE stands for **A**ddress **L**atch **E**nable. When ALE is set high, AD0 to AD7 works as an address bus; and when ALE is set low, AD0 to AD7 works as a data bus.

3. Describe the different addressing modes of 8085, 8086 microprocessors.

In 8085 microprocessor there are 5 types of addressing modes:

1. Immediate Addressing Mode –

In immediate addressing mode the source operand is always data. If the data is 8-bit, then the instruction will be of 2 bytes, if the data is of 16-bit then the instruction will be of 3 bytes.

Examples:

MVI B 45 (move the data 45H immediately to register B)

2. Register Addressing Mode –

In register addressing mode, the data to be operated is available inside the register(s) and register(s) is(are) operands. Therefore the operation is performed within various registers of the microprocessor.

Examples:

INR A (increment the contents of register A by one)

3. Direct Addressing Mode –

In direct addressing mode, the data to be operated is available inside a memory location and that memory location is directly specified as an operand. The operand is directly available in the instruction itself.

Examples:

LDA 2050 (load the contents of memory location into accumulator A)

4. Register Indirect Addressing Mode –

In register indirect addressing mode, the data to be operated is available inside a memory location and that memory location is indirectly specified by a register pair.

Examples:

LDAX B (move contents of B-C register to the accumulator)

STAX B (store accumulator contents in memory pointed by register pair B-C)

5. Implied/Implicit Addressing Mode –

In implied/implicit addressing mode the operand is hidden and the data to be operated is available in the instruction itself.

Examples:

RRC (rotate accumulator A right by one bit)

RLC (rotate accumulator A left by one bit)

6. Relative Addressing Mode:

In this mode, the operand is a memory location specified by the contents of the program counter plus a constant value.

example:

```
MOV R0,#05H
```

```
AGAIN:
```

```
MVI A,#55H
```

```
ADD A,R0
```

```
JMP AGAIN
```

In this example, the instruction JMP AGAIN uses the Relative Addressing Mode. The instruction jumps to the label AGAIN by adding the contents of the program

counter with the specified constant value. The constant value is calculated based on the distance between the current instruction and the label AGAIN.

In this case, the instruction JMP AGAIN is executing a loop that loads the accumulator with the value 55H, adds the contents of the register R0 to the accumulator, and then jumps back to the label AGAIN to repeat the loop.

Types of addressing modes in 8086 microprocessor:

Types of addressing modes:

1.Register mode – In this type of addressing mode both the operands are registers.

Example:

```
MOV AX, BX
```

2.Immediate mode – In this type of addressing mode the source operand is a 8 bit or 16 bit data. Destination operand can never be immediate data.

Example:

```
MOV AX, 2000
```

3.Displacement or direct mode – In this type of addressing mode the effective address is directly given in the instruction as displacement.

Example:

```
MOV AX, [DISP]
```

```
MOV AX, [0500]
```

4.Register indirect mode – In this addressing mode the effective address is in SI, DI or BX. $\text{Physical Address} = \text{Segment Address} + \text{Effective Address}$

Example:

```
MOV AX, [DI]
```

5. Based indexed mode – In this the effective address is sum of base register and index register.

Base register: BX, BP

Index register: SI, DI

The physical memory address is calculated according to the base register.

Example:

```
MOV AL, [BP+SI]
```

```
MOV AX, [BX+DI]
```

6. Indexed mode – In this type of addressing mode the effective address is sum of index register and displacement.

Example:

```
MOV AX, [SI+2000]
```

```
MOV AL, [DI+3000]
```

7. Based mode – In this the effective address is the sum of base register and displacement.

Example:

```
MOV AL, [BP+ 0100]
```

8. Based indexed displacement mode – In this type of addressing mode the effective address is the sum of index register, base register and displacement.

Example:

```
MOV AL, [SI+BP+2000]
```

9.String mode – This addressing mode is related to string instructions. In this the value of SI and DI are auto incremented and decremented depending upon the value of directional flag.

Example:

```
MOVS B
```

10.Input/Output mode – This addressing mode is related with input output operations.

Example:

```
IN A, 45
```

11.Relative mode –

In this the effective address is calculated with reference to instruction pointer.

Example:

```
JNZ 8 bit address
```

```
IP=IP+8 bit address
```

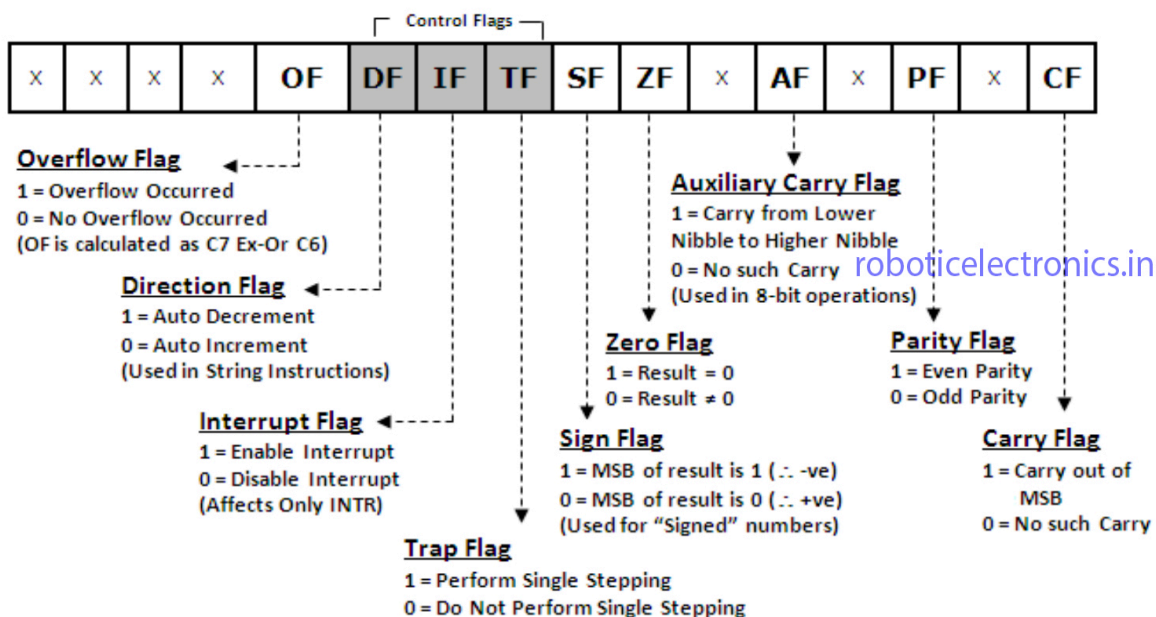
4. Draw and explain the flag structure of 8085 and 8086 microprocessor.

8085 FLAG REGISTER

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	X	AC	X	P	X	CY
SIGN	ZERO	AUXILIARY			PARITY		CARRY
FLAG	FLAG	CARRY			FLAG		FLAG

- **CY** - The carry flag is used for carrying and borrowing in case of addition and subtraction operations, it set when carry generated otherwise it reset
- **P** - The parity flag is used for results containing an even number of one's, it set for even parity and reset for odd parity
- **AC** - In an arithmetic operation, when a carry is generated by digit D3 and passed on to digit D4, the AC flag is set. The auxiliary carry flag is used for BCD operations, not free to the programmer
- **Z** - The zero flag is set if the result of an instruction is zero otherwise it reset
- **S** - Sign flag is -ve if S = 1 & Sign flag is +ve if S = 0

Flag structure of 8086 microprocessor:



5. Write a program to generate a 1msec, 1 sec and 10 msec time delay considers the operating frequency of 8085 is 2MHz and 3MHz.

➤ **NOP**

- no operation.
- Waste the processor time.

➤ T state is Fashionable name given to 1 complete clock cycle.

➤ If clock is 2 MHz: T state = $1/f = 1/2\text{MHz} = 0.5\mu\text{sec}$.

NOP- DELAY

OUT 01H

NOP

NOP

NOP

NOP

OUT 02H

$$\begin{aligned}\text{Time delay} &= (4T + 4T + 4T + 4T) \\ &= 16 \times 0.5\mu\text{sec} \\ &= 8\mu\text{sec}\end{aligned}$$

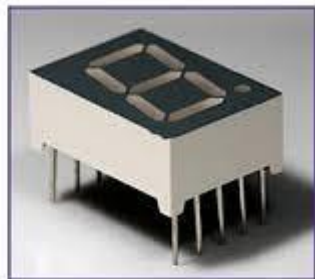
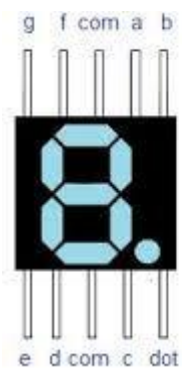
6. Differentiate between memory mapped I/O and peripheral mapped I/O or I/O mapped I/O. Draw the input and output interfacing diagram and display 0 to 9 to the seven segment display.

Memory mapped I/O	I/O mapped I/O
1. In this device address is 16 bit. Thus A_0 to A_{15} lines are used to generate device address.	1. In this I/O device address is 8 bit. Thus A_0 to A_7 or A_8 to A_{15} lines are used to generate device address.
2. $\overline{MEMR}$ and $\overline{MEMW}$ control signals are used to control read and write I/O operations.	2. $\overline{IOR}$ and $\overline{IOW}$ control signals are used to control read and write I/O operations.
3. Instructions available are LDA addr, STA addr, LDAX rp, STAX rp, MOV M,R, MOV R,M ADD M, CMP M etc.	3. Instructions available are IN and OUT.
4. Data transfer is between any register and I/O device.	4. Data transfer is between accumulator and I/O device.
5. Maximum number of I/O devices are 65536 (theoretically).	5. Maximum number of I/O devices are 256.
6. Execution speed using LDA addr, STA addr is 13 T-state and 7 T-states for MOV M, r and MOV r, M instructions.	6. Execution speed is 10 T-states.
7. Decoding 16 bit address may require more hardware.	7. Decoding 8 bit address will require less hardware.



Pin diagram of 8085 Microprocessor

A seven segment display is the most basic electronic display device that can display digits from 0–9. The most common configuration has an array of eight LEDs arranged in a special pattern to display these digits. They are laid out as a squared-off figure. Seven segment displays are of two types, common cathode and common anode. In common cathode type, the cathode of all LEDs are tied together to a single terminal and the anode of all LEDs are left alone as individual pins labeled as a, b, c, d, e, f, g & h (for dot). In common anode type, the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins. It will take a high (1) to turn a LED ON in common cathode and will take a low (0) to turn a LED ON in a common anode seven segment display. The pin out scheme and picture of a typical 7 segment LED display is shown in the image below.



Digit drive pattern for common cathode

a b c d e f g

0 1 1 1 1 1 0

1 0 1 1 0 0 0

2 1 1 0 1 1 0

3 1 1 1 1 0 0

4 0 1 1 0 0 1

5 1 0 1 1 0 1

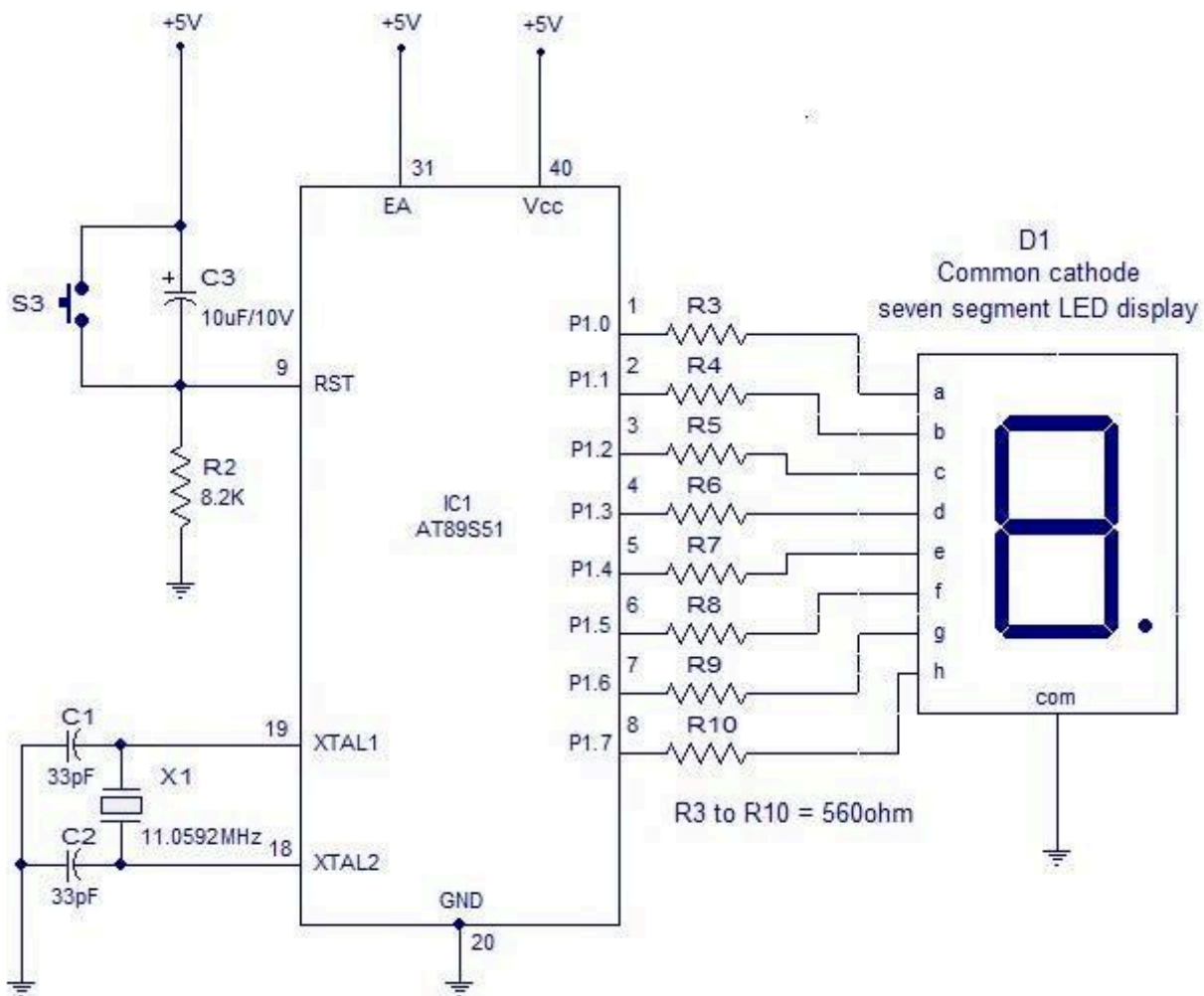
6 1 0 1 1 1 1

7 1 1 1 0 0 0

8 1 1 1 1 1 1

9 1 1 1 1 0 1

The circuit diagram shown below is of an AT89S51 microcontroller based 0 to 9 counter which has a 7 segment LED display interfaced to it in order to display the count. This simple circuit illustrates two things. How to setup simple 0 to 9 up counter using 8051 and more importantly how to interface a seven segment LED display to 8051 in order to display a particular result. The common cathode seven segment display D1 is connected to the Port 1 of the microcontroller (AT89S51) as shown in the circuit diagram. R3 to R10 are current limiting resistors. S3 is the reset switch and R2,C3 forms a debouncing circuitry. C1, C2 and X1 are related to the clock circuit. The software part of the project has to do the following tasks.



Form a 0 to 9 counter with a predetermined delay (around 1/2 second here).

Convert the current count into digit drive pattern.

Put the current digit drive pattern into a port for displaying.

All the above said tasks are accomplished by the program given below.

Program.

```
ORG 000H //initial starting address
```

```
START: MOV A,#00001001B // initial value of accumulator
```

```
MOV B,A
```

```
MOV R0,#0AH //Register R0 initialized as counter which counts from 10 to 0
```

```
LABEL: MOV A,B
```

```
INC A
```

```
MOV B,A
```

```
MOVC A,@A+PC // adds the byte in A to the program counters address
```

```
MOV P1,A
```

ACALL DELAY // calls the delay of the timer

DEC Ro//Counter Ro decremented by 1

MOV A,Ro // Ro moved to accumulator to check if it is zero in next instruction.

JZ START //Checks accu

7. Can the same port address be assigned to two I/O device if interfaced in peripheral mapped I/O? If yes, how and if no, Why? Can 8085 differentiate between memory and I/O device if the I/O device if the I/O is interfaced in memory mapped I/O? Explain.

In the context of the 8085 microprocessor:

Peripheral Mapped I/O:

Can the same port address be assigned to two I/O devices?

- No, the same port address cannot be assigned to two different I/O devices in peripheral mapped I/O. Each I/O device must have a unique port address.

How does the 8085 differentiate between memory and I/O devices in peripheral mapped I/O?

- The 8085 microprocessor uses a separate address space for memory and I/O. I/O devices are assigned specific port addresses, and when the CPU outputs an address to the address bus, the I/O devices respond only to their assigned addresses. The CPU uses specific instructions like `IN` and `OUT` to communicate with I/O devices through the assigned port addresses.

Memory Mapped I/O:

Can the same address be assigned to both memory and I/O devices in memory-mapped I/O?

- Yes, in memory-mapped I/O, both memory locations and I/O devices share the same address space. However, the distinction between memory and I/O operations is made using control signals.

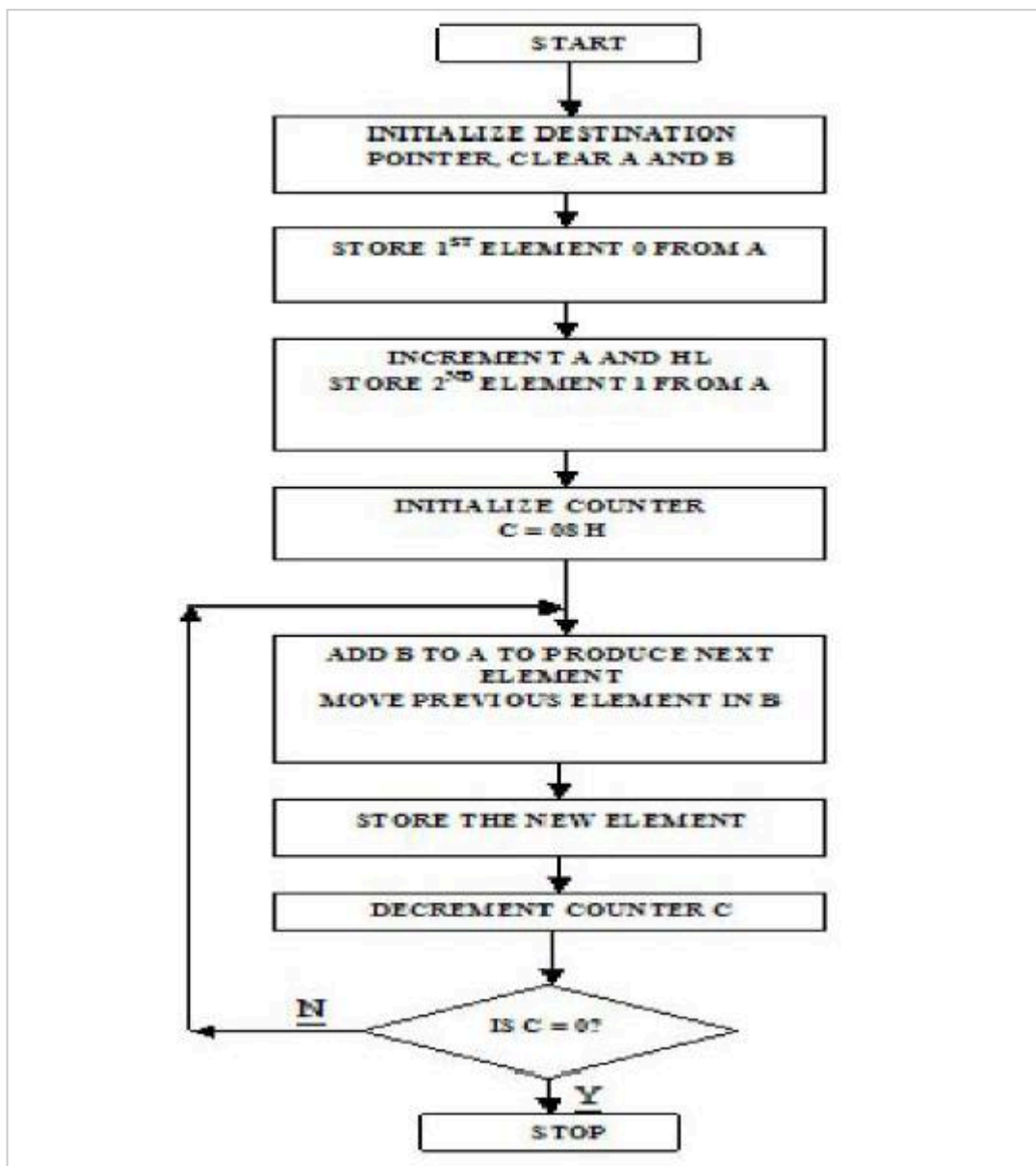
How does the 8085 differentiate between memory and I/O devices in memory-mapped I/O?

- The 8085 microprocessor relies on the Memory/IO (M/IO) control signal. When M/IO is low, it indicates a memory operation, and when it is high, it indicates an I/O operation. This control signal helps the CPU distinguish between accessing memory and communicating with I/O devices within the shared address space.

8. Write a program to generate the first eight elements of the Fibonacci series. The Fibonacci series is defined as $F(i) = F(i-1) + F(i-2)$, and $F(0) = 0$; $F(1) = 1$

In this case we are not providing any input, this program will generate ten Fibonacci numbers.

Flow Diagram



Program

Address	HEX Codes	Labels	Mnemonics	Comments
8000	21, 50, 80	START	LXI H 8050H	Pointer to the OUT-BUFFER
8003	AF		XRA A	Clear accumulator and reg. B
8004	47		MOV B, A	
8005	77		MOV M, A	Copying content to the target location
8006	3C		INR A	Increment A
8007	23		INX H	Go to the next dest. address.
8008	77		MOV M, A	Moving the content
80 09	0E, 08		MVI C, 08H	Initialize counter
800B	80	LOOP	ADD B	Getting the next term
800C	46		MOV B, M	Initializing term e.g. F1 = F2
800D	23		INX H	Go to the next dest. address.
800E	77		MOV M, A	Writing to the OUT-BUFFER
800F	0D		DCR C	Decrement count until 0 is reached F3= F1 + F2 (A) = (A) + (B) This is done with instruction ADDB.
8010	C2, 0B, 80		JNZ LOOP	
8013	76		HLT	Terminate the program

9. Explain the instruction of 8085 that uses auxiliary carry flag.

The 8085 microprocessor instruction that uses the Auxiliary Carry (AC) flag is the DAA instruction, which stands for "Decimal Adjust Accumulator." The DAA instruction is used to adjust the contents of the accumulator after an addition operation in BCD (Binary Coded Decimal) arithmetic.

In BCD arithmetic, each decimal digit is represented by a 4-bit binary code. After addition, the result in the accumulator might need adjustment to ensure it remains a valid BCD representation. The DAA instruction performs this adjustment, and the Auxiliary Carry (AC) flag is crucial in this process.

The DAA instruction takes into account the lower nibble (4 bits) of the accumulator and adjusts it if necessary. The AC flag indicates a carry-out from the low nibble during the previous addition operation.

The steps performed by the DAA instruction are as follows:

If the low nibble (bits 0-3) of the accumulator is greater than 9 or the AC flag is set, add 6 to the low nibble.

If the high nibble (bits 4-7) of the accumulator is greater than 9 or if there was a carry-out from the low nibble adjustment, add 6 to the high nibble.

The DAA instruction is particularly useful in applications involving BCD arithmetic, such as when working with decimal numbers in certain microcontroller applications or interfacing with decimal displays.

Example Assembly Code:

```
ORG 0000H
MOV A, 25H      ; Example BCD addition result in accumulator
DAA             ; Decimal Adjust Accumulator
HLT             ; Halt the program
```

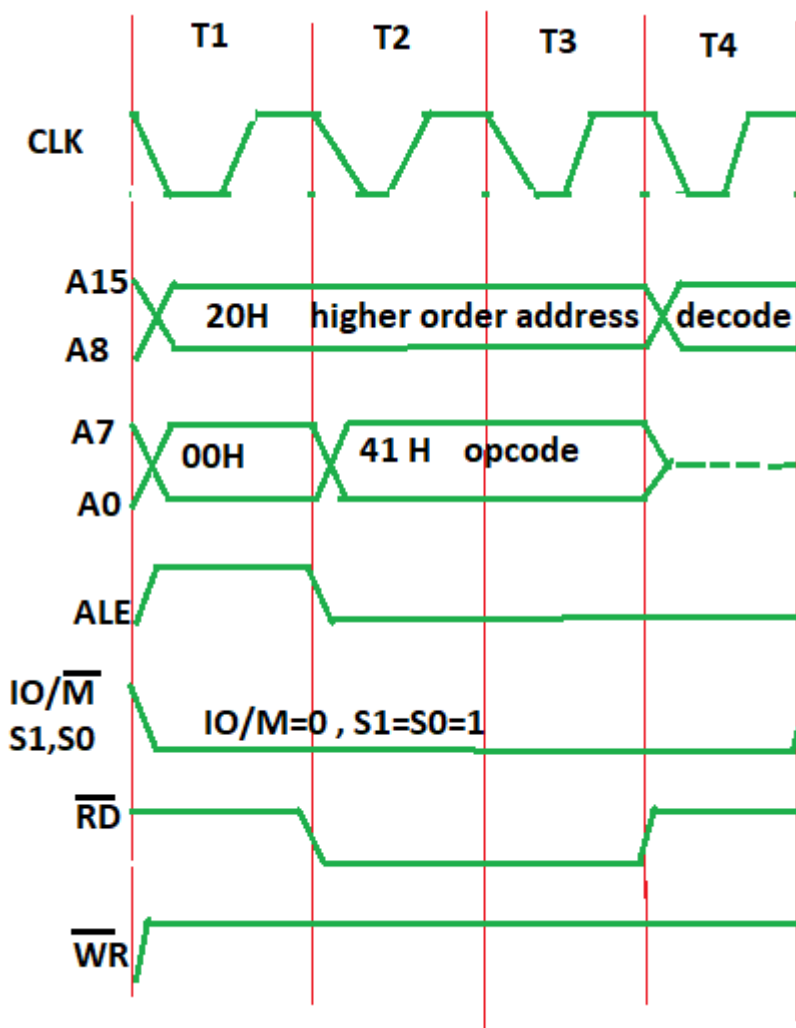
This example assumes that the accumulator contains a BCD result of a previous addition operation, and the DAA instruction adjusts the accumulator to ensure it represents a valid BCD result.

10. Draw the timing diagram for the instruction MOV A,M, INR M, ADD B, LHLD, IN, OUT, ORA M stored in memory location 9000H.

2000: MOV B, C

Only opcode fetching is required for this instruction and thus we need 4 T states for the timing diagram. For the opcode fetch the IO/M (low active) = 0, S1 = 1 and S0 = 1. The timing diagram of MOV instruction is shown below:

In Opcode fetch (t1-t4 T states):

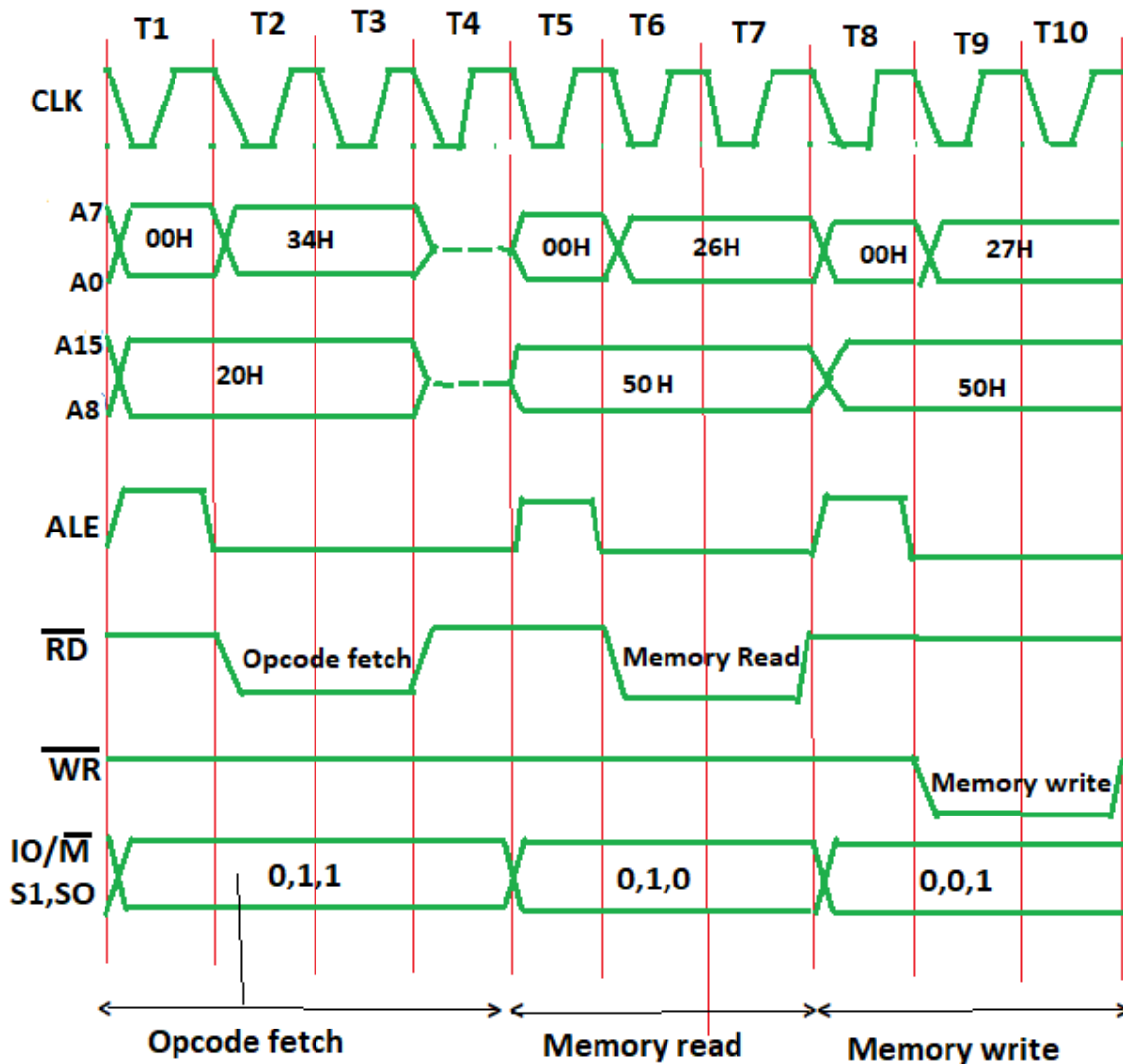


- 00 – lower bit of address where the opcode is stored, i.e., 00.
- 20 – higher bit of address where the opcode is stored, i.e., 20.

- **ALE** – provides signal for multiplexed address and data bus. Only in t1 is it used as an address bus to fetch a lower bit of address otherwise it will be used as the data bus.
- **RD (low active)** – signal is 1 in t1 & t4 as no data is read by the microprocessor. Signal is 0 in t2 & t3 because here the data is read by a microprocessor.
- **WR (low active)** – signal is 1 throughout, no data is written by a microprocessor.
- **IO/M (low active)** – signal is 1 throughout because the operation is performing on memory.
- **S0 and S1** – both are 1 in case of opcode fetching.

2000: INR M

The opcode fetch will be same as for other instructions in first 4 T states. Only the Memory read and Memory Write need to be added in the successive T states. For the opcode fetch the IO/M (low active) = 0, S1 = 1 and S0 = 1. For the memory read the IO/M (low active) = 0, S1 = 1 and S0 = 0. Also, only 3 T states will be required. For the memory write the IO/M (low active) = 0, S1 = 0 and S0 = 1 and 3 T states will be required. The timing diagram of INR M instruction is shown below:



In

Opcode fetch (t1-t4 T states) –

- **00**: lower bit of address where opcode is stored, i.e., 00
- **20**: higher bit of address where opcode is stored, i.e., 20.
- **ALE**: provides signal for multiplexed address and data bus. Only in t1 it used as address bus to fetch lower bit of address otherwise it will be used as data bus.
- **RD (low active)**: signal is 1 in t1 & t4 as no data is read by microprocessor. Signal is 0 in t2 & t3 because here the data is read by microprocessor.

- **WR (low active):** Signal is 1 throughout, no data is written by microprocessor.
- **IO/M (low active):** Signal is 0 in throughout because the operation is performing on memory.
- **S0 and S1:** both are 1 in case of opcode fetching.

In Memory read (t5-t7 T states) –

- **00:** lower bit of address where opcode is stored, i.e, 00
- **50:** higher bit of address where opcode is stored, i.e, 50.
- **ALE:** provides signal for multiplexed address and data bus. Only in t5 it used as address bus to fetch lower bit of address otherwise it will be used as data bus.
- **RD (low active):** signal is 1 in t5, no data is read by microprocessor. Signal is 0 in t6 & t7, data is read by microprocessor.
- **WR (low active):** signal is 1 throughout, no data is written by microprocessor.
- **IO/M (low active):** signal is 0 in throughout, operation is performing on memory.
- **S0 and S1 –** S1=1 and S0=0 for Read operation.

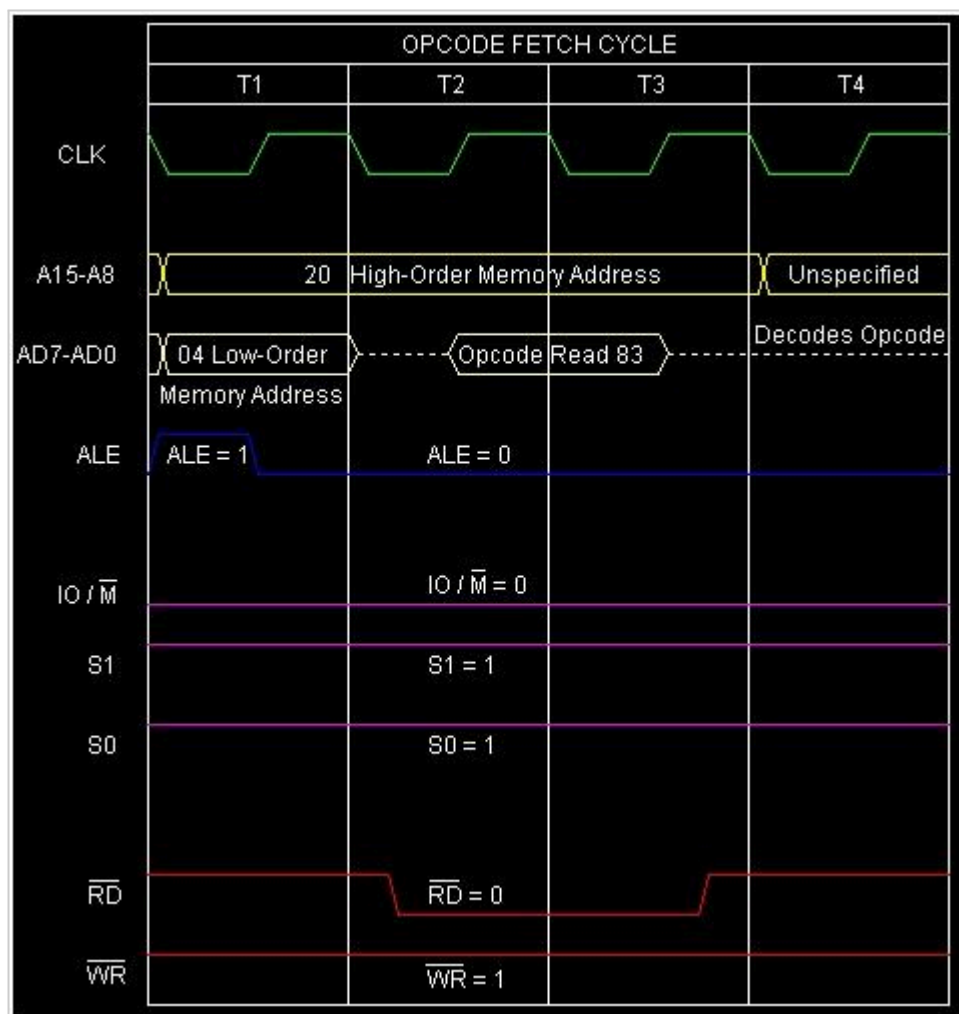
In Memory write (t8-t10 T states) –

- **00:** lower bit of address where opcode is stored, i.e, 00
- **50:** higher bit of address where opcode is stored, i.e, 50.

- **ALE:** provides signal for multiplexed address and data bus. Only in t8 it used as address bus to fetch lower bit of address otherwise it will be used as data bus.
- **RD (low active):** signal is 1 throughout, no data is read by microprocessor.
- **WR (low active):** signal is 1 in t8, no data is written by microprocessor. Signal is 0 in t9 & t10, data is written by microprocessor.
- **IO/M (low active):** signal is 0 in throughout, operation is performing on memory.
- S0 and S1 – S1=0 and S0=1 for write operation.

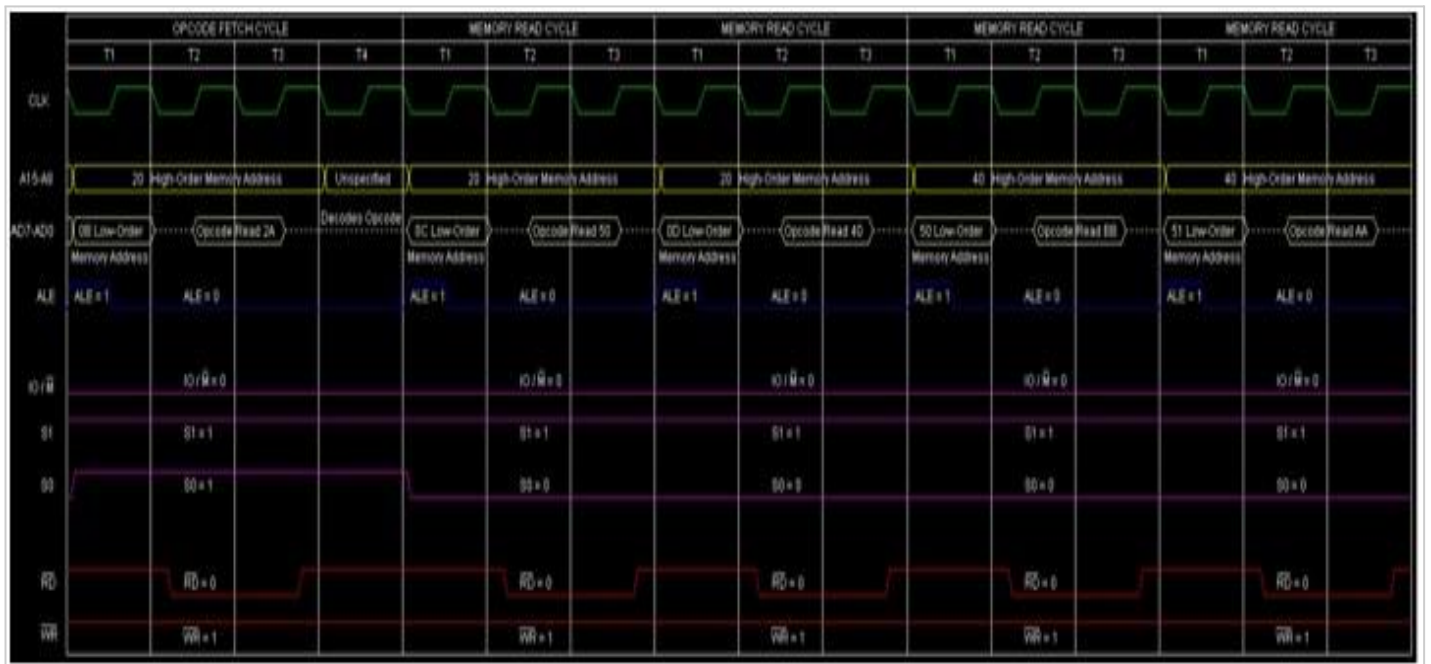
Address	Hex Codes	Mnemonic	Comment
2004	83	ADD E	Accumulator = Accumulator + Register E

Here is the timing diagram of the instruction **ADD E** execution.



Summary – So this instruction **ADD E** requires 1-Byte, 1 Machine Cycles (Opcode Fetch) and 4 T-States for execution

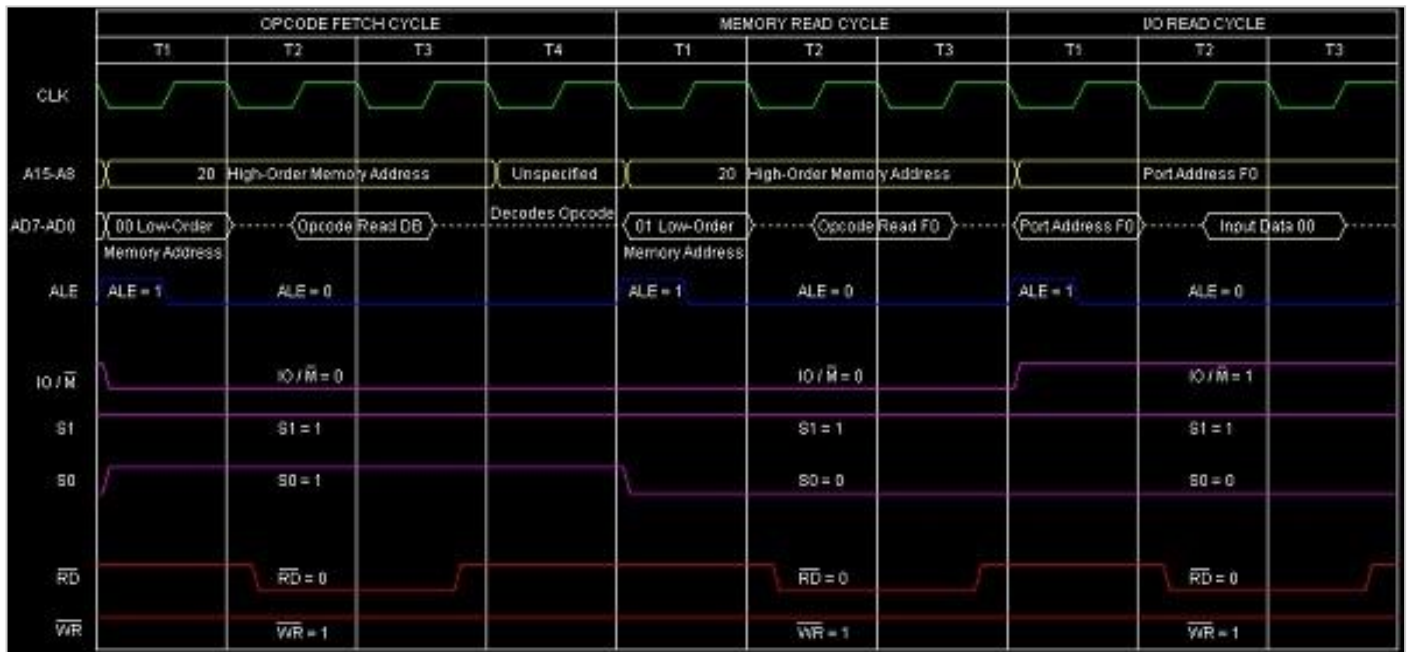
The timing diagram of this instruction **LHLD 4050H** below.



Summary – So this instruction **LHLD 4050H** requires 3-Bytes, 5-Machine Cycles (Opcode Fetch, Memory Read, Memory Read, Memory Read, Memory Read) and 16 T-States for execution

Address	Hex Codes	Mnemonic	Comment
2000	DB	IN F0H	Content from the port address F0H will be written back on to the Accumulator
2001	F0		F0H as port address

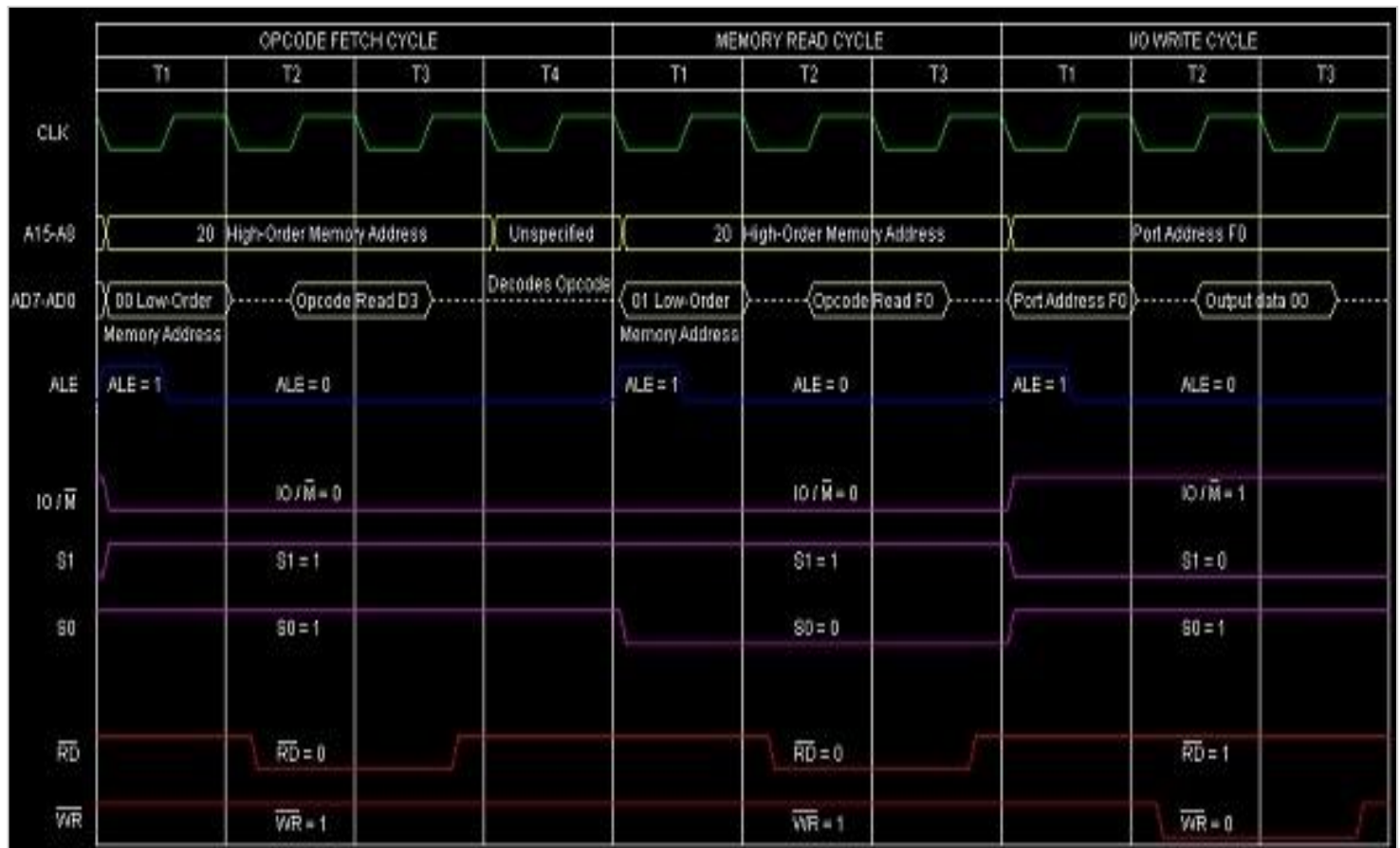
The timing diagram against this instruction IN F0H execution is as follows –



Summary – So this instruction OUT requires 2-Bytes, 3-Machine Cycle (Opcode Fetch, Memory Read, I/ORead) and 10 T-States for execution

Address	Hex Codes	Mnemonic	Comment
2000	D3	OUT F0H	Accumulator content will be sent to port address F0H
201	F0		F0H as port address

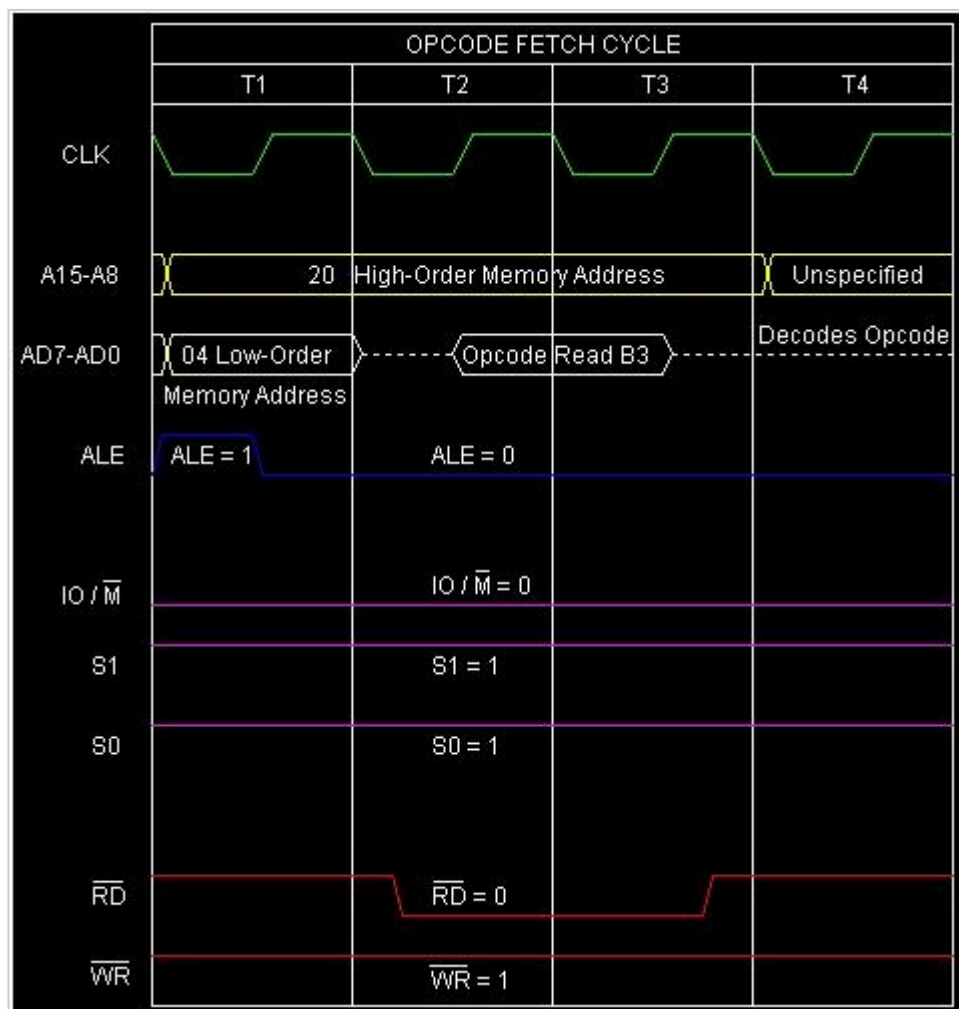
The timing diagram against this instruction OUT F0H execution is as follows –



Summary – So this instruction OUT requires 2-Bytes, 3-Machine Cycles (Opcode Fetch, Memory Read, I/O write) and 10 T-States for execution

Address	Hex Codes	Mnemonic	Comment
2004	B3	ORA E	A = A OR E

The timing diagram against this instruction **ORA E** execution is as follows –



Summary – So this instruction **ORA E** requires 1-Byte, 1-Machine Cycle (Opcode Fetch) and 4 T-States for execution

11. Explain the function of HOLD, HLDA, READY, RESET, S0, S1, ALE, CLK IN, CLK OUT

Focusing on the 8085 microprocessor:

HOLD (Hold Acknowledge):

- Function: It is an input signal used by external devices to request control of the address and data buses. When HOLD is activated, the 8085 relinquishes control of the buses to allow external devices to gain access.

HLDA (Hold Acknowledge):

- Function: HOLD Acknowledge is an output signal that indicates the acknowledgment of the HOLD request. When HLDA is active, it means the 8085 has granted control of the buses to the requesting external device.

READY:

- Function: An input signal used to synchronize the microprocessor with external devices. The microprocessor suspends its operation until the READY signal is active, ensuring that slower external devices have enough time to provide or fetch data.

RESET:

- Function: A signal that initializes the microprocessor and sets the program counter to zero. It is active low, meaning the microprocessor is reset when the signal is brought low.

S0 and S1 (Status Bits):

- Function: These are output signals indicating the status of the microprocessor. The combination of S0 and S1 bits represents the current state of the microprocessor, allowing external devices to understand the type of operation in progress (fetch, memory read, memory write, etc.).

ALE (Address Latch Enable):

- Function: A signal used to indicate that the address bus is about to latch valid address information. It typically occurs during the first clock cycle of each machine cycle, allowing external devices to sample the address bus.

CLK IN and CLK OUT (Clock Signals):

- CLK IN Function: An input signal that provides the clock pulses to the microprocessor, determining the rate at which it processes instructions.
- CLK OUT Function: An output signal that can be used to cascade multiple microprocessors or to synchronize other devices with the clock of the 8085.

12. Explain the function of ADD, SBI, LHLD, XCHG, CMP, ADI, CMC, STA, IN instruction.

In the context of the 8085 microprocessor:

ADD (Addition):

- Function: Adds the contents of the specified register or memory location to the contents of the accumulator. The result is stored in the accumulator, and the flags are updated based on the result.

SBI (Subtract Immediate with Borrow):

- Function: Subtracts an immediate value (8-bit) along with the borrow from the accumulator. The result is stored in the accumulator, and flags are set accordingly.

LHLD (Load HL from Memory):

- Function: Loads the contents of the memory location specified by the 16-bit address into the HL register pair. The next memory location is loaded into the L register, and the one after that into the H register.

XCHG (Exchange HL and DE):

- Function: Swaps the content of the HL register pair with the DE register pair. After execution, HL contains the original value of DE, and DE contains the original value of HL.

CMP (Compare):

- Function: Compares the contents of the accumulator with the specified register or memory location without changing the contents of the accumulator. It sets the flags based on the result of the comparison.

ADI (Add Immediate):

- Function: Adds an immediate 8-bit value to the accumulator. The result is stored in the accumulator, and flags are set based on the result.

CMC (Complement Carry):

- Function: Complements the carry flag. If the carry flag is set, it is cleared; if it is cleared, it is set. This instruction is often used in conjunction with other instructions to manipulate the carry flag.

STA (Store Accumulator):

- Function: Stores the contents of the accumulator in the memory location specified by the 16-bit address.

IN (Input):

- Function: Reads a byte from the input port specified by the immediate 8-bit data and loads it into the accumulator. The status of various flags may be affected based on the result.

13. What is interrupt? Explain different types of interrupts. What is RIM and SIM, and how it is execute? Write a program to enable all interrupt except RST 5.5.

Interrupt:

An interrupt is a mechanism that allows a microprocessor to temporarily halt its current program execution and handle a specific event or condition. Interrupts are essential for handling real-time events and improving the responsiveness of a system.

Types of Interrupts:

Hardware Interrupts:

- External Interrupts (INT): Triggered by external hardware devices.
- Internal Interrupts (TRAP): Non-maskable interrupt for critical events.

Software Interrupts:

- RST (Restart): Software-triggered interrupts with fixed addresses (RST 0-7).
- HLT (Halt): Executed when the HLT instruction is encountered.
- DI (Disable Interrupts) and EI (Enable Interrupts): Used for controlling interrupt enable/disable status.

RIM and SIM:

- RIM (Read Interrupt Mask): Reads the status of interrupt-related flags (including the interrupt enable/disable status) and loads them into the accumulator.
 - Format: `RIM`
- SIM (Set Interrupt Mask): Writes data from the accumulator into interrupt-related flags, allowing the user to set or reset the status of various interrupt-related control bits.
 - Format: `SIM`

Program to Enable All Interrupts Except RST 5.5:

```
ORG 0000H
START:
MVI A, 0FFH ; Load A with 11111111 (All interrupts enabled)
SIM ; Set interrupt mask based on the value in A
HLT ; Halt the program
```

In this program:

- `MVI A, 0FFH` loads the accumulator with the value 0xFF, which enables all interrupts.

- `SIM` sets the interrupt mask based on the value in the accumulator.
- `HLT` halts the program.

14. What do you mean by maskable and non maskable interrupts of 8085? Write a program to check whether RST 5.5 is pending. If it is pending, enable it without affecting the masking status of the other interrupts. Write a program to clear all flag of 8085 microprocessor.

Maskable and Non-Maskable Interrupts (8085):

- Maskable Interrupts:
 - Interrupts that can be temporarily disabled or masked by the user through specific instructions (DI - Disable Interrupts, EI - Enable Interrupts).
 - Examples include RST (Restart) interrupts (RST 0 to RST 7) and INTR (External Hardware Interrupt).
- Non-Maskable Interrupt (NMI):
 - An interrupt that cannot be disabled or masked by the user.
 - Example: TRAP, which is triggered by a trap or a non-maskable hardware interrupt.

Program to Check and Enable RST 5.5 if Pending:

```
ORG 0000H
```

```
START:
```

```
RIM ; Read Interrupt Mask to check the status of interrupts
```

```
CPI 0020H ; Check if RST 5.5 is pending (bit 5)
```

```
JZ RST_PENDING ; Jump if pending
```

```
HLT ; Halt if not pending
```

```
RST_PENDING:
```

```
MVI A, 0020H ; Load A with 00100000 (RST 5.5 mask)
```

```
SIM ; Set RST 5.5 interrupt without affecting other interrupts
```

```
HLT ; Halt the program
```

In this program:

- `RIM` reads the interrupt-related flags into the accumulator.
- `CPI 0020H` compares the accumulator with the value `0020H` (binary `00100000`) to check if the RST 5.5 interrupt is pending.
- If RST 5.5 is pending (`JZ RST_PENDING`), it sets RST 5.5 interrupt without affecting other interrupts using `MVI A, 0020H` and `SIM`.

Program to Clear All Flags of 8085 Microprocessor:

```
ORG 0000H
```

```
START:
```

```
SIM ; Set all interrupt-related flags to 0
```

```
HLT ; Halt the program
```

In this simple program, `SIM` is used to set all interrupt-related flags to 0, effectively clearing them. The program then halts. Note that this approach clears the flags without affecting the interrupt enable/disable status.

15. Interface the 4KB EPROM, 2 KB RAM and 4 KB EPROM memory devices in 8085 microprocessor in absolute decoding method.

Memory Interfacing in 8085:

Memory is an integral part of a microprocessor system, and in this section, we will discuss how to interface a memory device with the microprocessor. The Memory Interfacing in 8085 is used to access memory quite frequently to read instruction codes and data stored in memory. This read/write operations are monitored by control signals. The microprocessor activates these signals when it wants to read from and write into memory. In the last section we have already seen the memory read and memory write machine cycles, and status of the RD, WR and IO/M status signals for read/write operation. In the following section we will see memory structure and its requirements, concepts in Memory Interfacing in 8085 and interfacing examples.

Memory Structure and its Requirements:

As mentioned earlier, read/write memories consist of an array of registers, in which each register has unique address. The size of the memory is $N \times M$ as shown in Fig. 4.13 (a) where N is the number of registers and M is the word length, in number of bits.

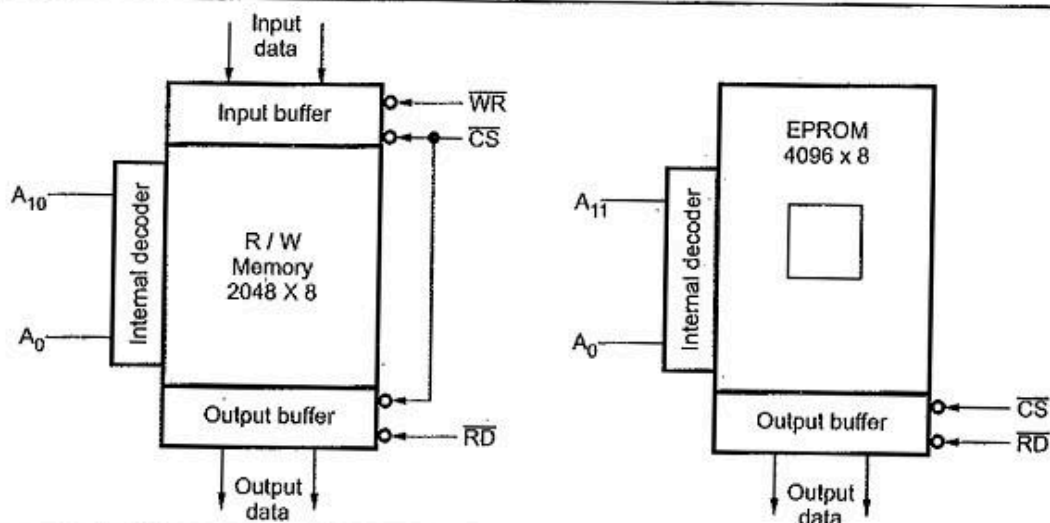


Fig. 4.13 (a) Logic diagram for RAM Fig. 4.13 (b) Logic diagram for EPROM

Basic Concepts in Memory Interfacing:

For Memory Interfacing in 8085, following important points are to be kept in mind.

1. Microprocessor 8085 can access 64Kbytes memory since address bus is 16-bit. But it is not always necessary to use full 64Kbytes address space. The total memory size depends upon the application.
2. Generally EPROM (or EPROMs) is used as a program memory and RAM (or RAMs) as a data memory. When both, EPROM and RAM are used, the total address space 64Kbytes is shared by them.
3. The capacity of program memory and data memory depends on the application.
4. It is not always necessary to select 1 EPROM and 1 RAM. We can have multiple EPROMs and multiple RAMs as per the requirement of application.
5. We can place EPROM/RAM anywhere in full 64 Kbytes address space. But program memory (EPROM) should be located from address 0000H since reset address of 8085 microprocessor is 0000H.
6. It is not always necessary to locate EPROM and RAM in consecutive memory. For example : If the mapping of EPROM is from 0000H to 0FFFH, it is not must to locate RAM from 1000H. We can locate it anywhere between 1000H and FFFFH. Where to locate memory component totally depends on the application.

The memory interfacing requires to :

- **Select the chip**
- **Identify the register**
- **Enable the appropriate buffer.**

Microprocessor system includes memory devices and I/O devices. It is important to note that microprocessor can communicate (read/write) with only one device at a time, since the data, address and control buses are common for all the devices. In order to communicate with memory or I/O devices, it is necessary to decode the address from the microprocessor. Due to this each device (memory

or I/O) can be accessed independently. The following section describes common address decoding techniques.

Address Decoding Techniques :

- **Absolute decoding/Full Decoding**
- **Linear decoding/Partial Decoding**

Absolute decoding:

In absolute decoding technique, all the higher address lines are decoded to select the memory chip, and the memory chip is selected only for the specified logic levels on these high-order address lines; no other logic levels can select the chip. Fig. 4.14 shows the Memory Interfacing in 8085 with absolute decoding. This addressing technique is normally used in large memory systems.

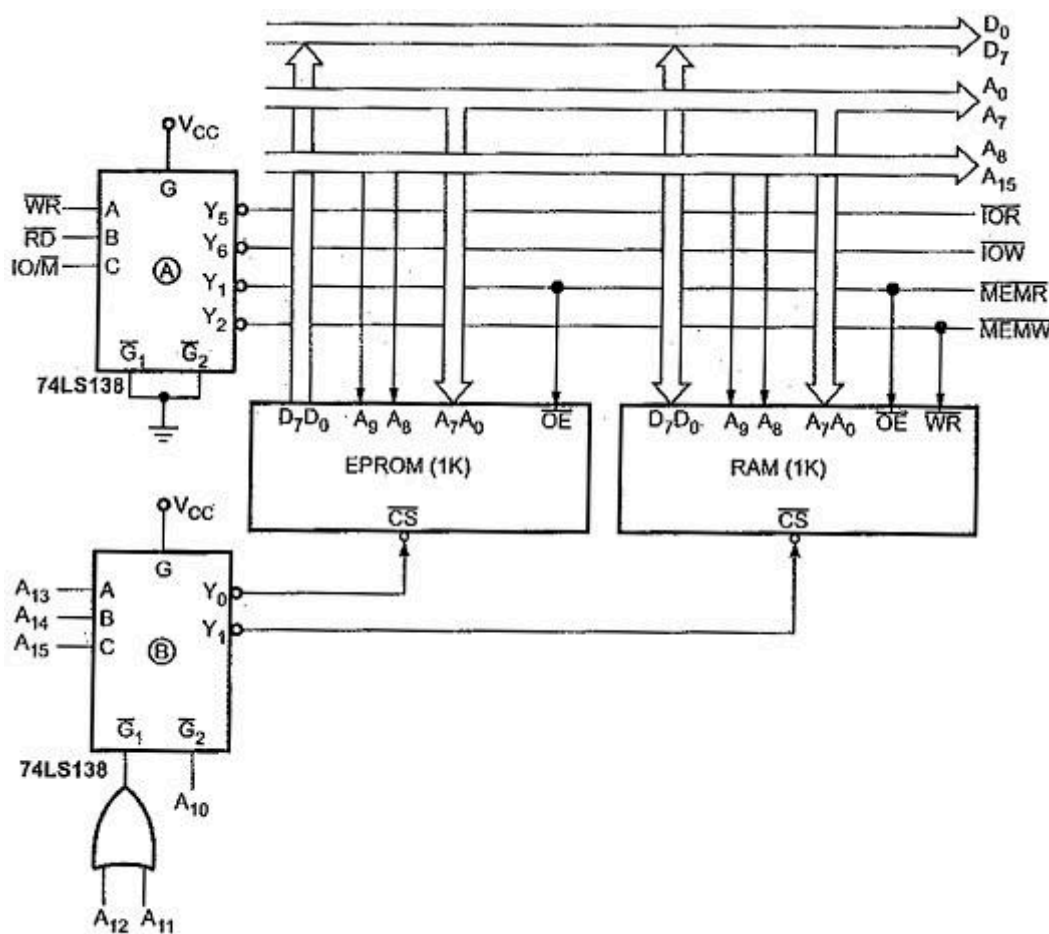


Fig. 4.14 Absolute decoding technique

Linear decoding:

In small systems, hardware for the decoding logic can be eliminated by using individual high-order address lines to select memory chips. This is referred to as linear decoding. Fig. 4.15 shows the addressing of RAM with linear decoding technique. This technique is also called **partial decoding**. It reduces the cost of decoding circuit, but it has a drawback of multiple addresses (shadow addresses).

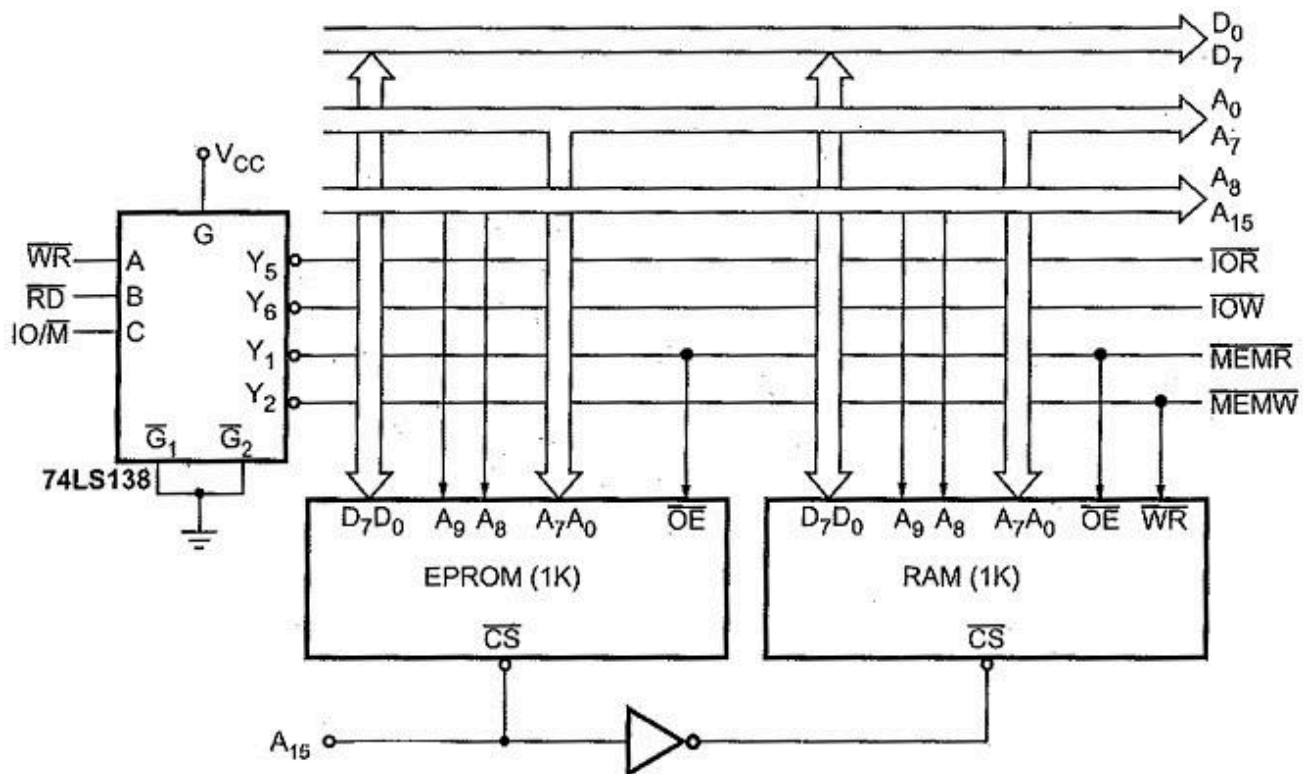


Fig. 4.15 Linear decoding

Fig. 4.15 shows the addressing of RAM with linear decoding technique. A_{15} address line, is directly connected to the chip select signal of EPROM and after inversion it is connected to the chip select signal of the RAM. Therefore, when the status of A_{15} line is 'zero', EPROM gets selected and when the status of A_{15} line is 'one' RAM gets selected. The status of the other address lines is not considered, since those address lines are not used for generation of chip select signals.

Wait State Generator:

We have seen that wait states can be introduced in the machine cycles using READY signal to interface slower memory devices. In this section we are going to

see the circuits for wait state generator and applications of those circuits to interface slower memories.

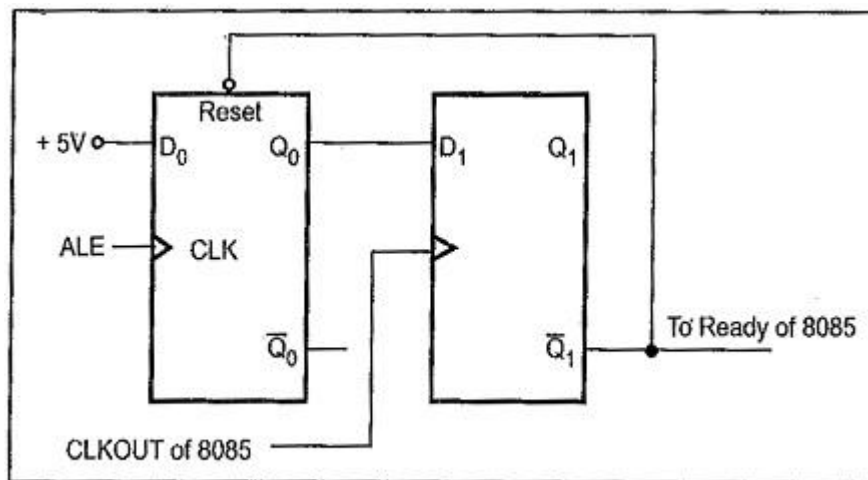


Fig. 4.22 Wait state generator

The Fig. 4.22 shows the circuit for wait state generator. This circuit generates single wait state in every machine cycle. The circuit operation is given below.

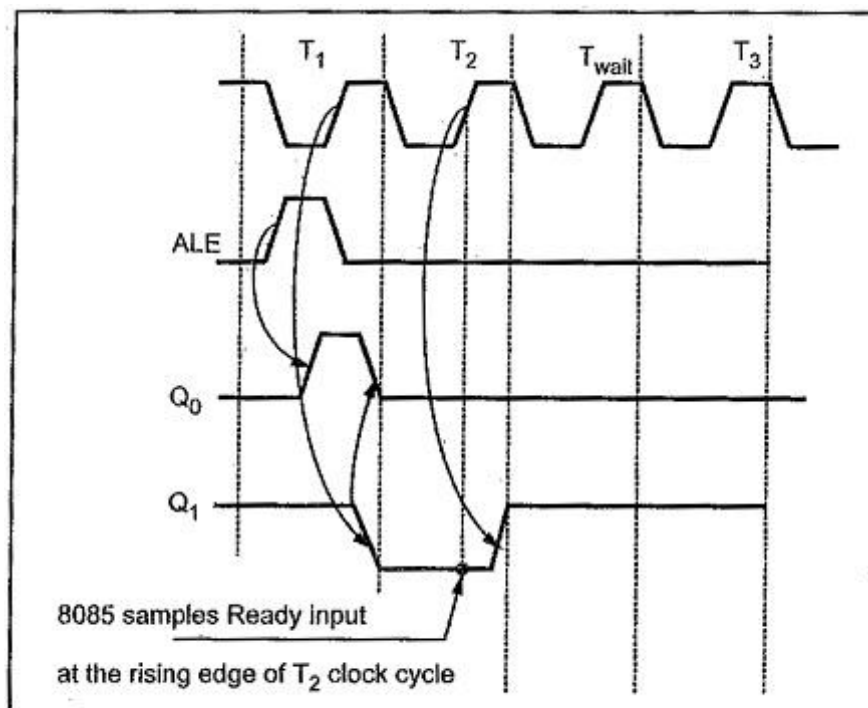


Fig. 4.23 Timing waveforms for wait state generator circuit

In T_1 , the ALE signal is activated, which makes output Q_0 of the first D flip-flop high, assuming RESET (R) is one. In the next clock pulse i.e. in the next low to high transition of CLKOUT signal of the 8085, the output Q_1 of the second D

flip-flop goes low. The low on the Q₁ output initiates 8085 to enter wait state and resets the first D flip-flop making its Q₀ output low, since Q₁ is connected to the ready input of 8085 and to the reset input of the first D flip-flop. At next low to high transition of CLKOUT, Q of the second D flip-flop goes high making READY pin high and inactivating reset input of the first flip-flop. Fig. 4.23 shows the timing waveforms for this circuit.

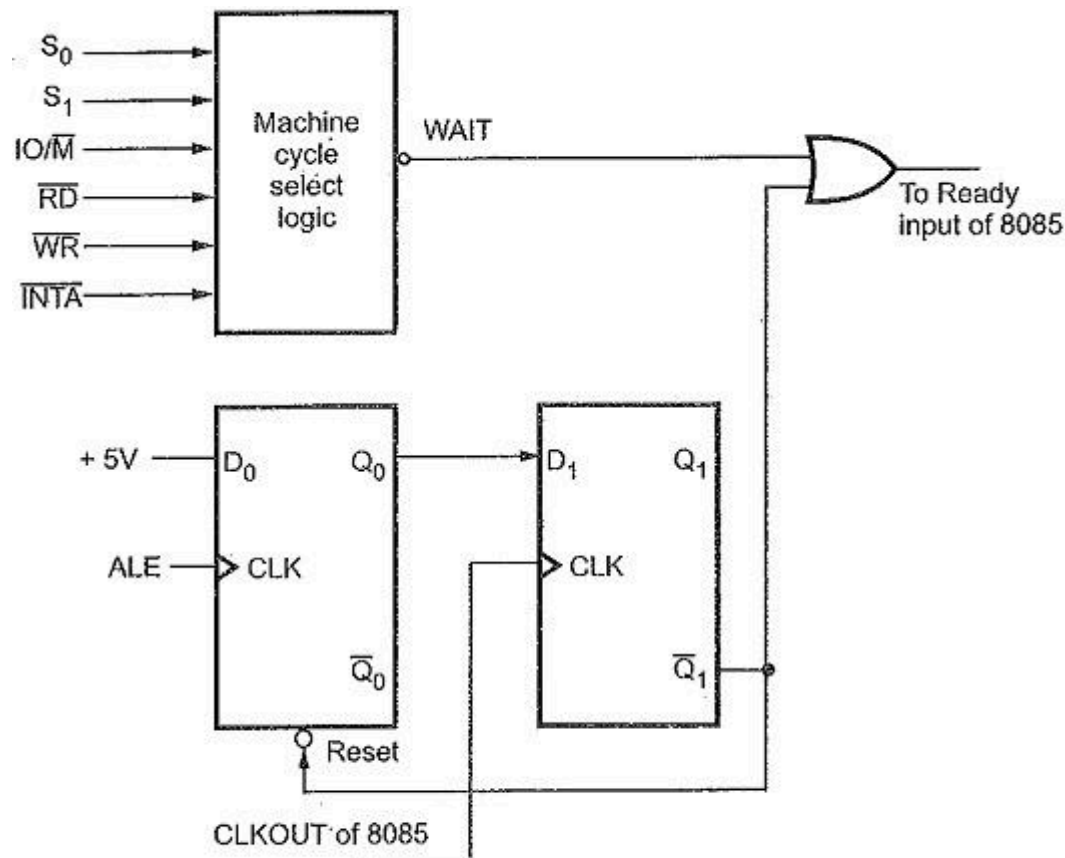


Fig. 4.24 Modified Wait State Generator Circuit

This circuit can be modified to introduce wait states for a particular machine cycle. The modified circuit of wait state generator is shown . The modified circuit will generate a single WAIT state for any machine **cycle** for which the WAIT output of the machine cycle select circuit is logic 0.

16. CALL and RET are the only instructions of 8085 that uses subroutine. Comment on this statement. PUSH and POP are the only instructions of 8085 that uses Stack. Comment on this statement. Write a program to exchange the contents of DE register pair with the HL register pair using PUSH and POP instruction. Write a program to clear the all flags without effecting the content of accumulator.

CALL and RET for Subroutines:

While CALL and RET are commonly used in subroutine calls, they are not the only instructions related to subroutines in the 8085 microprocessor. The 8085 also has other instructions like PUSH and POP, which are often used to store and retrieve data during subroutine calls. Additionally, instructions like JMP (jump) and conditional branch instructions can also be used in subroutine implementation.

PUSH and POP for Stack:

Similar to the first statement, PUSH and POP are indeed instructions used for stack operations, but they are not the only instructions related to the stack in the 8085 microprocessor. The stack-related instructions also include CALL, RET, and various conditional branch instructions that manipulate the stack during subroutine calls and returns.

Now, let's provide programs for the tasks you mentioned:

Exchange DE register pair with HL register pair using PUSH and POP:

```
; Assuming DE and HL contain values to be exchanged
```

```
MOV A, D ; Copy D to accumulator
```

```
MOV D, H ; Copy H to D
```

```
MOV H, A ; Copy accumulator to H
```

```
MOV A, E ; Copy E to accumulator
```

```
MOV E, L ; Copy L to E
```

```
MOV L, A ; Copy accumulator to L
```

This program uses temporary storage in the accumulator to exchange the contents of DE and HL register pairs.

Clear all flags without affecting the content of the accumulator:

```
MOV A, A ; No operation, just to use accumulator
```

```
; Clear all flags
```

```
MOV C, A ; Clear carry flag
```

```
MOV P, A ; Clear parity flag
```

```
MOV AC, A ; Clear auxiliary carry flag
```

```
MOV Z, A ; Clear zero flag
```

```
MOV S, A ; Clear sign flag
```

In this program, the accumulator is not affected, and each flag is individually cleared.

17. Explain the process of segmentation of memory in 8086 microprocessor. What are its advantages? Draw and explain the flag register of 8086 microprocessor. Explain how to calculate 20 bit physical address. How pipelining achieved in 8086 microprocessor.

Segmentation in 8086 Microprocessor:

In the 8086 microprocessor, memory segmentation is a process that divides the memory into segments. Each segment can be up to 64 KB in size, and the microprocessor uses a combination of a 16-bit segment register and a 16-bit offset to generate a 20-bit physical address. The memory segmentation in 8086 allows addressing a total of 1 MB of memory (2^{20} bytes).

The segmentation process involves multiplying the segment register by 16 and adding the offset to generate a physical address. The formula for calculating the physical address is:

Physical Address=(Segment Register×16)+Offset

Physical Address=(Segment Register×16)+Offset

Advantages of Segmentation:

- **Memory Management:** Segmentation allows the efficient management of memory by dividing it into logical segments.
- **Flexibility:** Segmentation provides flexibility in organizing program and data segments, enabling modular programming.
- **Protection:** Segmentation enables protection mechanisms by assigning different access permissions to different segments.
- **Code Sharing:** Segmentation facilitates code sharing by allowing multiple programs to share common code segments.

Flag Register in 8086 Microprocessor:

The 8086 microprocessor has a 16-bit flag register (FLAGS) that contains various status and control flags. The flags in the register are:

- **Zero Flag (Z):** Set if the result of an operation is zero.
- **Sign Flag (S):** Reflects the sign of the result (negative or positive).
- **Parity Flag (P):** Indicates whether the number of set bits in the result is even or odd.
- **Carry Flag (C):** Set if there is a carry-out or borrow into the most significant bit.
- **Auxiliary Carry Flag (AC):** Used for binary-coded decimal (BCD) arithmetic.
- **Overflow Flag (V/O):** Indicates whether signed arithmetic overflow has occurred.
- **Direction Flag (D):** Determines the direction for string operations (forward or backward).
- **Interrupt Flag (I):** Enables or disables interrupts.
- **Trap Flag (T):** Causes a single-step interrupt after the execution of each instruction for debugging.
- **Reserved (R):** Reserved for future use.
- **Parity Flag (P):** Reflects the parity of the least significant byte of the result.

Calculating 20-Bit Physical Address:

As mentioned earlier, the physical address in the 8086 microprocessor is calculated using the formula:

Physical Address=(Segment Register×16)+Offset

Physical Address=(Segment Register×16)+Offset

Given that the segment register and offset are both 16-bit values, the result is a 20-bit physical address.

Pipelining in 8086 Microprocessor:

Pipelining in the 8086 microprocessor is achieved through the use of instruction pipelining. The pipeline stages in the 8086 include instruction prefetch, instruction decode, operand fetch, and execution. Each stage is responsible for a specific part of the instruction execution.

In a pipelined architecture, while one instruction is being executed, the next instruction can be fetched and decoded. This overlapping of instruction execution improves the overall throughput of the processor. Pipelining helps in achieving higher instruction execution rates and enhances the efficiency of the processor.

However, it's essential to note that the 8086 microprocessor has a simple pipeline with limited stages compared to modern processors. The effectiveness of pipelining is influenced by the nature of the instruction mix and the dependencies among instructions.

18. Draw and discuss in brief the BIU and EU of 8086 microprocessor. What is data and address size in 8086? How its physical address is calculated? What is pipelining architecture of 8086 microprocessor? Explain the function of Queue register, AX, BX, CX and DX register.

Bus Interface Unit (BIU) and Execution Unit (EU) of 8086:

- Bus Interface Unit (BIU):
 - Responsible for interacting with the external world.
 - Manages the bus operations, including fetching instructions and data from memory.
 - Contains the Segment Register, Instruction Pointer (IP), and the Address Generation Unit (AGU).
- Execution Unit (EU):
 - Executes the instructions fetched by the BIU.
 - Consists of the Arithmetic and Logic Unit (ALU) and the General Registers (AX, BX, CX, DX, SI, DI, SP, BP).
 - Performs mathematical and logical operations on data.

Data and Address Size in 8086:

- Data Size: 8086 is a 16-bit microprocessor, meaning it processes data in 16-bit chunks.
- Address Size: Although the internal architecture of the 8086 is 16-bit, it uses a segmentation mechanism that allows it to address up to 1 MB of memory. The

physical address is calculated using a 20-bit address formed by combining a 16-bit segment address and a 16-bit offset address.

Physical Address Calculation:

Physical Address=(Segment Register×16)+Offset

Physical Address=(Segment Register×16)+Offset

This formula illustrates how the 20-bit physical address is generated from the 16-bit segment address and 16-bit offset address.

Pipelining Architecture of 8086 Microprocessor:

The 8086 microprocessor features a simple instruction pipeline, consisting of two stages:

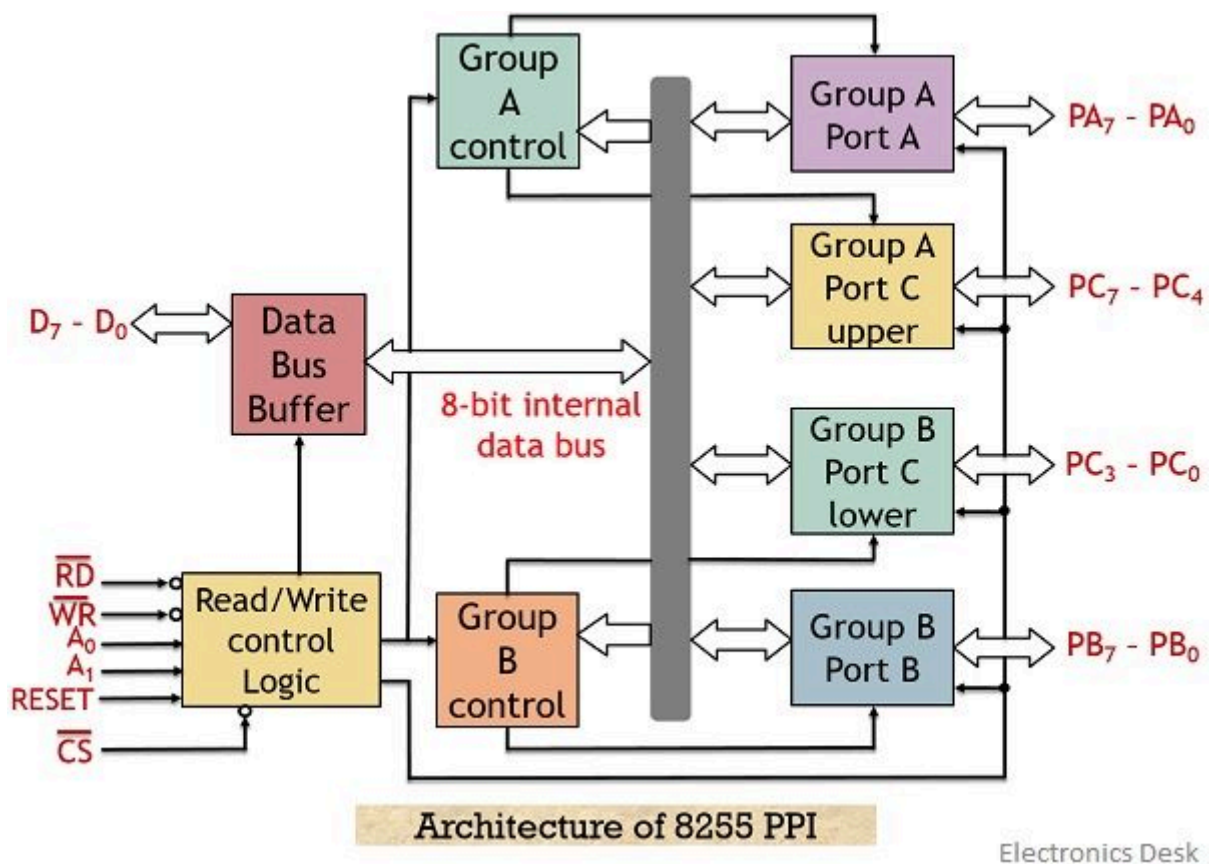
- Instruction Prefetch: Fetches the next instruction from memory.
- Instruction Execution: Decodes and executes the current instruction.

This two-stage pipeline allows for some degree of instruction overlap and helps in improving the overall performance by reducing the instruction cycle time.

Registers in 8086 and their Functions:

- Queue Register: The Queue register is not explicitly mentioned as part of the 8086 architecture. It is possible that you are referring to the Queue in the BIU, which is a temporary storage area for pre-fetched instructions.
- AX, BX, CX, and DX Registers:
 - AX (Accumulator Register): Often used for arithmetic and logic operations. It is also the implicit destination in many instructions.
 - BX (Base Register): Can be used as a base register for memory addressing.
 - CX (Count Register): Primarily used as a loop counter in iterative operations.
 - DX (Data Register): Has various uses, such as holding the high-order result in multiplication and division.

19. Draw and explain the functional block diagram of 8255. Draw and explain the control word format of 8255. Write a control word for port A and port C as an input port and port B as an output port. Explain mode 1 operation of 8255 when used as an Input and output port with proper diagrams. Explain the BSR mode of 8255. Write a control word for PC7 set and PC5 reset.



8255 is a programmable I/O device that acts as interface between peripheral devices and the [microprocessor](#) for parallel data transfer. 8255 PPI (programmable peripheral interface) is programmed in a way so as to have transfer of data in different conditions according to the need of the system.

In 8255, 24 pins are assigned to the I/O ports. Basically it has three, 8-bit ports that are used for simple or interrupt I/O operations.

The three ports are **Port A**, **Port B** and **Port C** and as each port has 8 lines, but the 8 bits of port C is divided into 2 groups of 4-bit each. These are given as port C lower i.e., $PC_3 - PC_0$ and port C upper i.e., $PC_7 - PC_4$. And are arranged in group of 12 pins each thus designated as Group A and Group B.

Data bus buffer: It is used to connect the internal bus of 8255 with the system bus so as to establish proper interfacing between the two. The data bus buffer allows the read/write operation to be performed from/to the CPU.

The buffer allows the passing of data from ports or control register to CPU in case of write operation and from CPU to ports or status register in case of read operation.

Read/ Write control logic: This unit manages the internal operations of the system. This unit holds the ability to control the transfer of data and control or status words both internally and externally.

Whenever there exists a need for data fetch then it accepts the address provided by the processor through the bus and immediately generates command to the 2 control groups for the particular operation.

Group A and Group B control: These two groups are handled by the CPU and functions according to the command generated by the CPU. The CPU sends control words to the group A and group B control and they in turn sends the appropriate command to their respective port.

As we have discussed that group A has the access of the port A and higher order bits of port C. While group B controls port B with the lower order bits of port C.

CS: It stands for chip select. A low signal at this pin shows the enabling of communication between the 8255 and the processor. More specifically we can say that the data transfer operation gets enabled by an active low signal at this pin.

RD – It is the signal used for read operation. A low signal at this pin shows that CPU is performing read operation at the ports or status word. Or we can say that 8255 is providing data or information to the CPU through data buffer.

WR – It shows write operation. A low signal at this pin allows the CPU to perform write operation over the ports or control register of 8255 using the data bus buffer.

A₀ and A₁: These are basically used to select the desired port among all the ports of the 8255 and it do so by forming conjunction with RD and WR. It forms connection with the LSB of the address bus.

The table below shows the operation of the control signals:

A_1	A_0	RD'	WR'	CS'	Input/Output Operation
0	0	0	1	0	Port A - Data Bus
0	1	0	1	0	Port B - Data Bus
1	0	0	1	0	Port C - Data Bus
0	0	1	0	0	Data Bus - Port A
0	1	1	0	0	Data Bus - Port B
1	0	1	0	0	Data Bus - Port C
1	1	1	0	0	Data Bus - Control register

Reset: It is an active high signal that shows the resetting of the PPI. A high signal at this pin clears the control registers and the ports are set in the input mode.

Initializing the ports to input mode is done to prevent circuit breakdown. As in case of reset condition, if the ports are initialized to output mode then there exist chances of destruction of 8255 along with the processor.

The 8255 microprocessor is an input/output (I/O) device that can be used to interface with various peripheral devices. It has three operating modes: mode 0, mode 1, and mode 2.

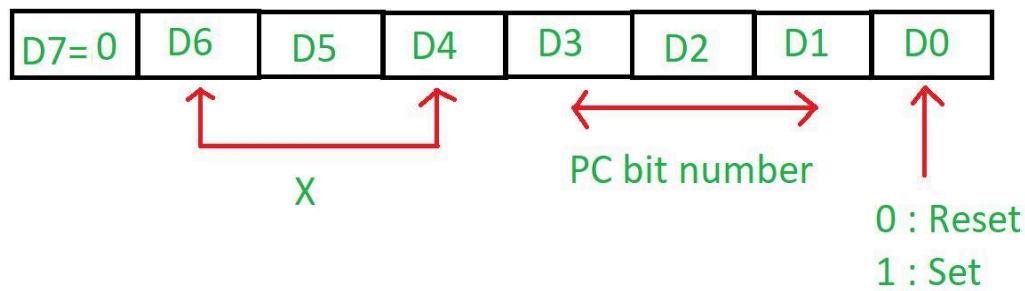
Mode 0 is the basic input/output mode. In this mode, the 8255 can be used as an 8-bit input or output device, with each of the three ports (Port A, Port B, and Port C) being configured individually as input or output.

Mode 1 is the strobed input/output mode. This mode is similar to mode 0, but includes additional control signals to enable latching of input data and/or output data.

Mode 2 is the bidirectional mode. In this mode, Port A and Port B are configured as bidirectional I/O ports, while Port C is configured as a control port for setting various operational modes.

Each of these operating modes can be further customized through programming of various control registers, allowing for flexible and efficient interfacing with a wide range of peripheral devices.

There are 2 modes in 8255 microprocessors: **1. Bit set reset (BSR) mode** – This mode is used to set or reset the bits of port C only, and selected when the most significant bit (D7) in the control register is 0. Control Register is as follows:



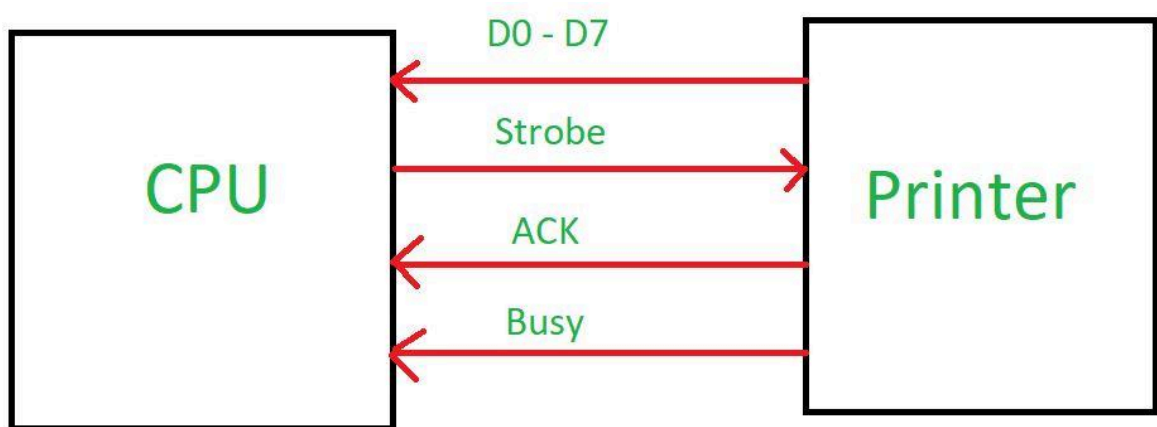
D3	D2	D1
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

This mode affects only one bit of port C at a time because, as user set the bit, it remains set until and unless user changes it. User needs to load the bit pattern in control register to change the bit. **2. Input/output mode (I/O)** – This mode is selected when the most significant bit (D7) in the control register is 1.

- **Mode 0 – Simple or basic I/O mode:** Port A, B and C can work either as input function or as output function. The outputs are latched but the inputs are not latched. It does not have interrupt handling capability.
- **Mode 1 – Handshake or strobed I/O:** In this either port A or B can work and port C bits are used to provide handshaking. The outputs as well as

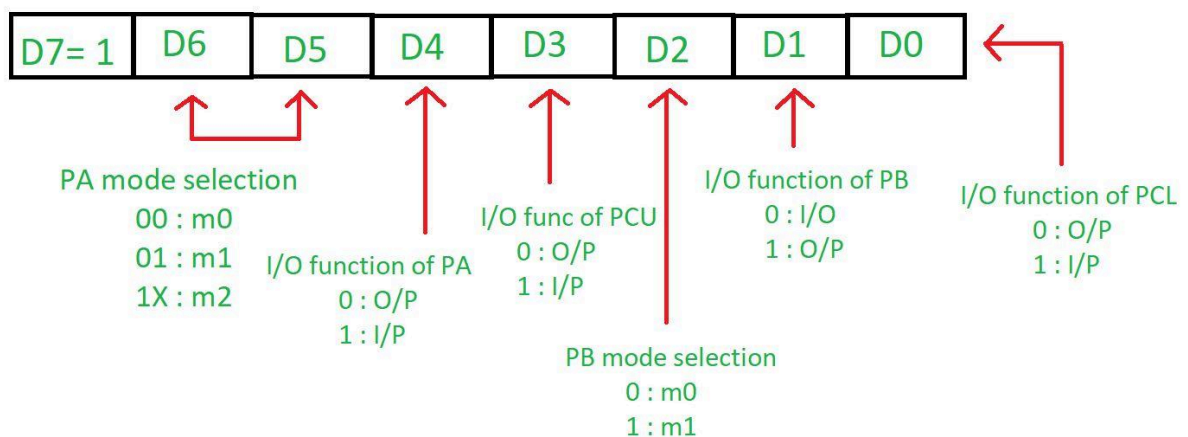
inputs are latched. It has interrupt handling capability. Before actual data transfer there is transmission of signal to match speed of CPU and printer.

Example: When CPU wants to send data to a slow peripheral device like printer, it will send handshaking signal to printer to tell whether it is ready or not to transfer the data. When the printer is ready it will send one acknowledgement to CPU then there will be transfer of data through data bus.



- **Mode 2 – Bidirectional I/O:** In this mode only port A will work, port B can either be in mode 0 or 1 and port C bits are used as handshake signal. The outputs as well as inputs are latched. It has interrupt handling capability.

Control Register is as follows:



The most significant bit (**D7**) is 1 for the I/O mode and 0 for the BSR mode.

D6 & D5 It is used to set the port A mode.

D6 D5 Mode

0	0	m0
0	0	m1
0	X	m2

D4 is used to tell whether port A is taking input or displaying the result. If it is 1 then it is taking input otherwise displaying output. **D3** is used to tell whether port C higher bits is taking input or displaying the result. If it is 1 then it is taking input otherwise displaying output. **D2** tells the mode of port B. If it is 0 then port B is in m0 mode otherwise in m1 mode. **D1** is used to tell whether port B is taking input or displaying the result. If it is 1 then it is taking input otherwise displaying output. **D0** is used to tell whether port C lower bits is taking input or displaying the result. If it is 1 then it is taking input otherwise displaying output.

When 8255 microprocessor is reset, it will clear the control word register contents, setting all the ports to input mode.

20. Draw and explain the DMA operation.

DMA, or Direct Memory Access, is a feature in computer systems that enables peripherals to transfer data directly to and from memory without involving the CPU. This process enhances overall system performance by offloading data transfer tasks from the CPU. Let's discuss the DMA operation and illustrate it with a simple diagram:

DMA Operation:

Initialization:

- The DMA controller is initialized by the CPU with the source and destination addresses, transfer count, and control signals.

Request:

- A peripheral device requests DMA by sending a request signal to the DMA controller.

DMA Controller Response:

- The DMA controller receives the request, and if granted, it temporarily suspends the CPU from accessing the system bus.

Bus Arbitration:

- The DMA controller performs bus arbitration to take control of the system bus.

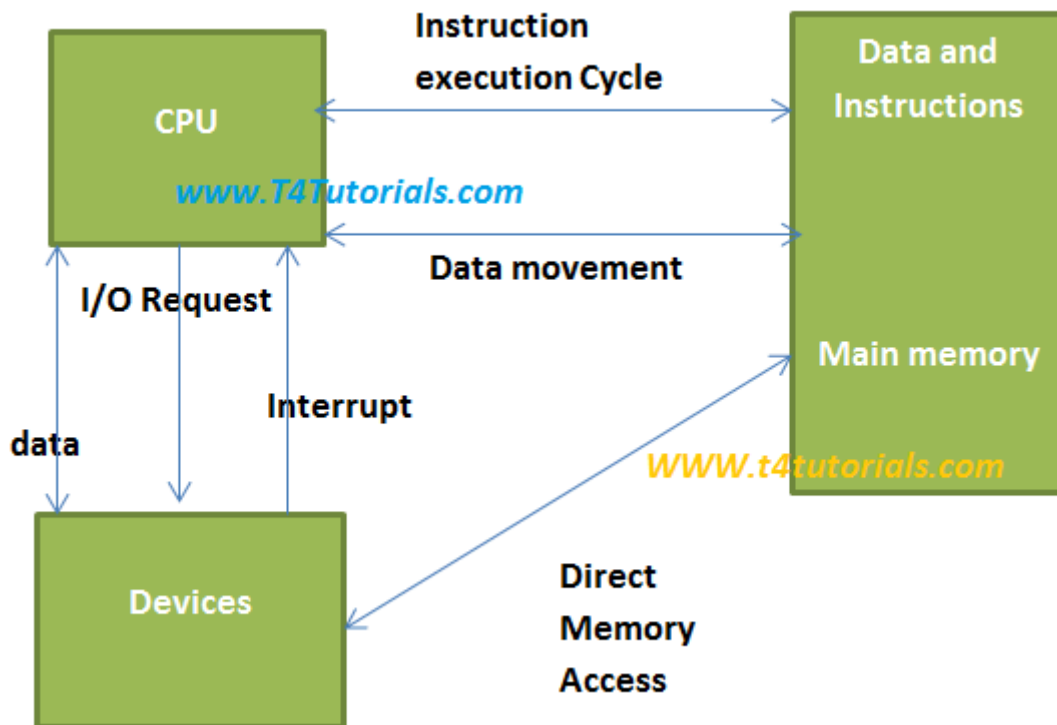
Transfer:

- The DMA controller transfers data directly between the peripheral device and memory without CPU involvement.

Interrupt or Polling:

- Upon completion of the data transfer, the DMA controller generates an interrupt signal or sets a status bit for the CPU to resume control.

Here is a simple representation of the DMA operation:



Explanation:

- The CPU initializes the DMA controller with the necessary parameters.
- The peripheral device requests DMA.
- The DMA controller, upon receiving the request, gains control of the system bus through bus arbitration.
- Data transfer occurs directly between the peripheral device and memory.
- Once the transfer is complete, the DMA controller signals the CPU (via interrupt or status bit), allowing it to regain control of the system bus.

Advantages of DMA:

- Increased Throughput: DMA allows for faster data transfer between peripherals and memory, improving overall system throughput.
- Reduced CPU Overhead: The CPU is free to perform other tasks while data transfer is handled by the DMA controller.
- Efficient Data Movement: Ideal for block data transfers and high-speed I/O devices.

DMA is commonly used in scenarios such as disk I/O, network interfaces, and graphics processing, where large amounts of data need to be moved between peripherals and memory efficiently.

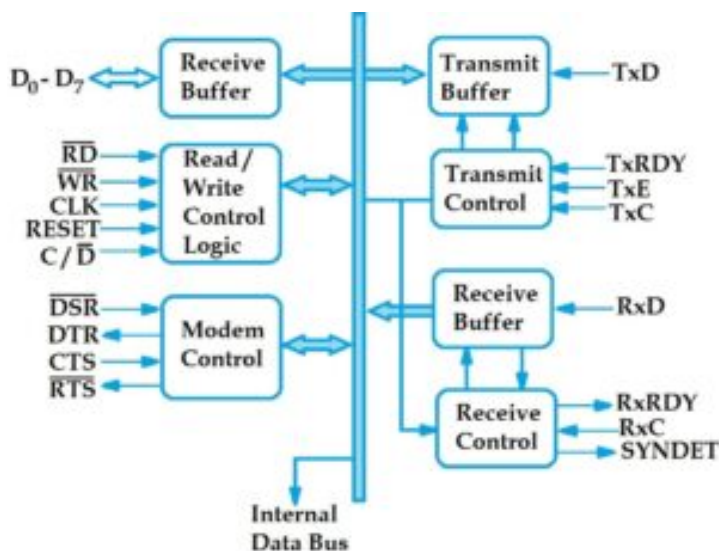
21. Draw and explain the 8251 USART and 8254 PIT operation.

The 8251 USART (Universal Synchronous/Asynchronous Receiver/Transmitter) and 8254 Programmable Interval Timer (PIT) are programmable peripheral devices commonly used in microprocessor-based systems. Here, I'll provide a brief explanation of the operation of both devices.

8251 USART (Universal Synchronous/Asynchronous Receiver/Transmitter):

The 8251 USART is a communication device that facilitates the serial transmission and reception of data between a microprocessor and external devices. It can operate in both synchronous and asynchronous modes.

Block Diagram of 8251 USART:



Operation:

Control Register:

- The control register configures the USART for various parameters such as word length, mode selection, and interrupt enable/disable.

Baud Rate Generator:

- Generates the clock pulses for the communication by dividing the system clock frequency.

Transmit and Receive Registers:

- These registers hold the data to be transmitted or the received data.

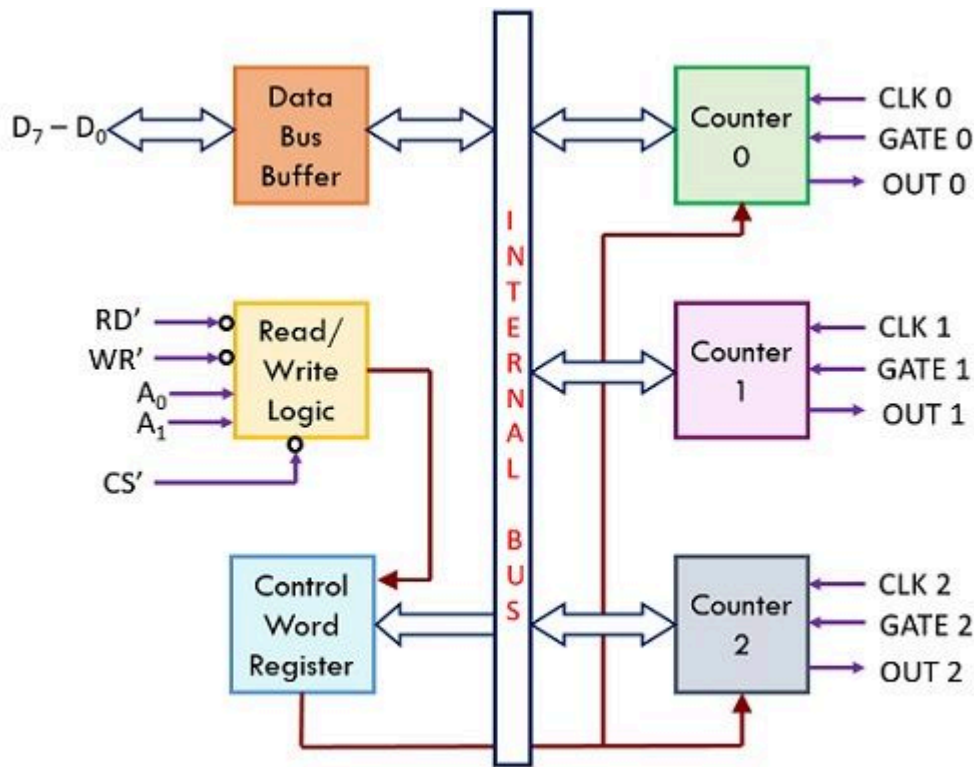
Data Buffer Register:

- Serves as a buffer for data during transmission and reception.

8254 Programmable Interval Timer (PIT):

The 8254 PIT is a timer device used for generating accurate time delays and periodic interrupts.

Block Diagram of 8254 PIT:



Functional Block Diagram of 8254 Timer

Operation:

Counters:

- The 8254 has three independent counters (Counter 0, Counter 1, and Counter 2). Counter 0 and Counter 1 are 16-bit, and Counter 2 is 8-bit.

Control Register:

- It is used to configure the operating mode (one-shot, rate generator, square wave generator, etc.) for each counter.

Clock Input:

- The 8254 is driven by an external clock signal.

Output Pins:

- Each counter has an output pin whose state is controlled by the configured operating mode.

Interrupts:

- The PIT can be configured to generate interrupts when a count reaches a specific value.

These devices play crucial roles in the interfacing of microprocessors with external devices, providing capabilities for serial communication and precise timing control. Keep in mind that the exact configurations and operations may vary based on specific implementations and applications.

22. Write a 8085 assembly language program to compare two strings. Assume that the first byte of both strings contain the number of bytes in that string. The starting address of the two strings is 9000h and 9100h respectively. If both strings are found equal place 11h in memory location 9500h else place 22h.

```
ORG 8000H ; Starting address
```

```
MOV C, 9000H ; Load the address of the first string
```

```
MOV B, M ; Get the length of the first string
```

```
MOV H, B ; Move the length to H register
```

```
INX C ; Point to the actual characters in the first string
```

```
MOV D, 9100H ; Load the address of the second string
```

```
MOV E, M ; Get the length of the second string
```

```
CMP B ; Compare the lengths of the two strings
```

```
JZ equal ; If lengths are equal, continue with character comparison
```

```
; If lengths are not equal
```

MOV A, 22H ; Set A register to 22H (not equal)

STA 9500H ; Store the result in memory location 9500H

HLT ; Halt the program

equal:

MOV A, H ; Move the length of the string to A register

; Loop to compare characters in the strings

compare_loop:

MOV A, M ; Load character from the first string

CMP E ; Compare it with the character from the second string

JNZ not_equal ; If not equal, jump to not_equal

INX C ; Move to the next character in the first string

INX D ; Move to the next character in the second string

DCX H ; Decrement the length counter

JZ strings_equal ; If length counter is zero, strings are equal

JMP compare_loop ; Continue the comparison loop

not_equal:

MOV A, 22H ; Set A register to 22H (not equal)

STA 9500H ; Store the result in memory location 9500H

```
HLT ; Halt the program
```

```
strings_equal:
```

```
MOV A, 11H ; Set A register to 11H (equal)
```

```
STA 9500H ; Store the result in memory location 9500H
```

```
HLT ; Halt the program
```

In this program, the lengths of both strings are compared first. If the lengths are equal, it proceeds to compare each character of the two strings. If a mismatch is found at any point, it sets the result to 22H (not equal). If the lengths are not equal, it also sets the result to 22H (not equal). If the entire loop completes without finding any mismatches and the lengths are equal, it sets the result to 11H (equal). The final result is stored in memory location 9500H.

23. Write a program to separate the positive and negative numbers in a series of 8 data bytes. The series starts from 9000H. Store the positive numbers from 9100H and negative numbers from 9200H.

Assembly language program for the 8085 microprocessor that separates positive and negative numbers in a series of 8 data bytes. The series starts from memory location 9000H. Positive numbers are stored starting from 9100H, and negative numbers are stored starting from 9200H.

```
ORG 9000H ; Set the origin address to 9000H
```

```
MVI C, 08H ; Set the counter to 8 (number of data bytes)
```

```
LXI H, 9000H ; Load the starting address of the series
```

```
MOV B, H ; Initialize B register with the series starting address
```

```
AGAIN:
```

```
MOV A, M ; Load the number from memory into the accumulator
```

```
JZ DONE ; If the number is zero, exit the loop
```

```
CMP 00H ; Compare with zero to check the sign
```

```
JZ SKIP ; If zero, skip storing
```

```
JNC POSITIVE ; If carry is not set, it's a positive number
```

```
JMP NEGATIVE ; If carry is set, it's a negative number
```

POSITIVE:

MOV A, M ; Move the positive number to the accumulator

MOV D, B ; Copy the address to D register

MVI E, 00H ; Set the offset for positive numbers

STAX D ; Store the positive number at the positive address

JMP CONTINUE

NEGATIVE:

MOV A, M ; Move the negative number to the accumulator

MOV D, B ; Copy the address to D register

MVI E, 20H ; Set the offset for negative numbers

STAX D ; Store the negative number at the negative address

CONTINUE:

INX H ; Increment the source address

INX B ; Increment the counter and destination address

DCR C ; Decrement the counter

JNZ AGAIN ; If counter is not zero, repeat the loop

DONE: HLT ; Halt the program

SKIP: INX H ; Skip storing and increment the source address

INX B ; Increment the counter and destination address

DCR C ; Decrement the counter

```
JNZ AGAIN ; If counter is not zero, repeat the loop
```

```
HLT ; Halt the program
```

24. Six 8-bit data is stored from memory location C070H. Transfer the entire block of data to new memory locations starting from C050H.

An 8085 assembly language program that transfers a block of six 8-bit data from memory location C070H to new memory locations starting from C050H:

```
ORG 8000H ; Starting address
```

```
MOV C, C070H ; Load source address (C070H) to register C
```

```
MOV D, C050H ; Load destination address (C050H) to register D
```

```
MOV B, 06H ; Set the loop counter to 6 (since there are six 8-bit data)
```

```
transfer_loop:
```

```
MOV A, M ; Load data from the source address
```

```
MOV M, A ; Store the data to the destination address
```

```
INX C ; Increment the source address
```

```
INX D ; Increment the destination address
```

```
DCR B ; Decrement the loop counter
```

```
JNZ transfer_loop ; Continue the loop until the counter is not zero
```

```
HLT ; Halt the program
```

In this program:

- `MOV C, C070H` loads the starting address of the source block (C070H) into register C.
- `MOV D, C050H` loads the starting address of the destination block (C050H) into register D.
- `MOV B, 06H` sets the loop counter to 6, as there are six 8-bit data to be transferred.
- The loop (`transfer_loop`) loads the data from the source address, stores it to the destination address, increments both the source and destination addresses, and decrements the loop counter. The loop continues until the counter becomes zero.
- `HLT` halts the program.

After executing this program, the six 8-bit data originally stored at C070H will be transferred to new memory locations starting from C050H.